AHALIA SCHOOL OF ENGINEERING & TECHNOLOGY
ISO 9001:2015 Certified Institution. Approved by AICTE & Affiliated to A. P. J. Abdul Kalam Technological University
Ahalia Health, Heritage & Knowledge Village, Palakkad - 678557 Ph: 04923-226666 www.ahalia.ac.in
Accredited by NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL

Department of Computer Science and Engineering



Laboratory Manual

PCCSL508 MACHINE LEARNING LAB

Based on 2024 Scheme of Kerala Technological University Syllabus

B.Tech. Fifth Semester Computer Science & Engineering

Prepared by

Mr. Krishnan M N
Lab in charge

Verified by

Dr.S. Gunasekaran
H.O.D. (CSE)

COURSE OUTCOMES

CO	Description	Level	Bloom's Level
CO1	Understand the complexity of Machine Learning algorithms and their limitations.	K2	Understanding
CO2	Understand modern notions in data analysis-oriented computing.	K2	Understanding
CO3	Apply common Machine Learning algorithms in practice and implement their own.	K3	Applying
CO4	Perform experiments in Machine Learning using real-world data.	K3	Applying

SL NO	EXPERIMENTS
1	Implement linear regression with one variable on the California Housing data set to predict housing prices based on a single feature (eg. the average number of rooms per dwelling). Tasks: • Load and preprocess the dataset. • Implement linear regression using both gradient descent and normal equation. • Evaluate the model performance using metrics such as Mean Squared Error (MSE) and R-squared. • Visualize the fitted line along with the data points.
2	Implement polynomial regression on the Auto MPG dataset to predict miles per gallon (MPG) based on engine displacement. Compare polynomial regression results with linear regression. Tasks: • Load and preprocess the dataset. • Implement polynomial regression of varying degrees. • Compare the polynomial regression model with linear regression using metrics such as MSE and R-squared. • Visualize the polynomial fit.
3	Implement Ridge and Lasso regression on the Diabetes dataset. Compare the performance of these regularized models with standard linear regression. Tasks: • Load and preprocess the dataset. • Implement Ridge and Lasso regression. • Tune hyper parameters using cross-validation. • Compare performance metrics (MSE and R-squared) with standard linear regression.
4	Estimate the parameters of Logistic Regression model using MLE and MAP on the Breast Cancer Wisconsin dataset. Compare the results and discuss the effects of regularization. Tasks: • Load and preprocess the dataset. • Implement logistic regression with MLE. • Apply MAP estimation with different regularization priors (L1 and L2 regularizations). • Compare performance and parameter estimates with MLE and MAP.
5	Use MLE and MAP to estimate the parameters of multinominal distribution on the 20 Newsgroups dataset. Explore the impact of different priors on the estimation. Tasks: • Load and preprocess the dataset. • Implement MLE for the multinominal distribution parameter estimation. • Apply MAP estimation with various priors (eg. Dirichelt priors). Compare the results and evaluate the effect of different priors.
6	Implement logistic regression model to predict the likelihood of a disease using the Pima Indians Diabetes dataset. Compare the performance with and without feature scaling. Tasks: • Load and preprocess the Pima Indians Diabetes dataset. • Implement logistic regression for binary classifications. • Evaluate model performance with and without feature scaling. • Analyze metrics such as accuracy, precision, recall, and F1-score.

7	Implement a Naïve Bayes classifier to categorize text documents into topics using the 20 Newsgroups dataset. Compare the performance of Multinominal Naïve Bayes with Bernoulli Naïve Bayes. Tasks: • Load and preprocess the 20 Newsgroups dataset. • Implement Multinomial Naïve Bayes and Bernoulli Naïve Bayes. • Evaluate and compare the performance of both models using metrics such as accuracy and F1 score. Discuss the strength and weakness of each Naïve Bayes variant for text classification.
8	Implement the K-Nearest Neighbors (KNN) algorithm for image classification using the Fashion MNIST dataset. Experiment with different values of K and analyze their impact on model performance. Tasks: • Load and preprocess the Fashion MNIST dataset. • Implement KNN for multi-class classification. • Experiment with different values of K and evaluate performance. • Discuss the impact of different K values on model accuracy and computational efficiency.
9	Implement a Decision Tree classifier using ID3 algorithm to segment customers based on their purchasing behavior using the Online Retail dataset. Analyze the tree structure and discuss the feature importance. Tasks: • Load and preprocess the Online Retail dataset. • Implement Decision Tree using ID3 algorithm. • Visualize the decision tree and analyze the feature importance. • Discuss the tree structure helps in understanding customer behavior.
10	Implement and compare Logistic Regression and Decision Tree on the Adult Income dataset for predicting income levels. Evaluate both models based on performance metrics and interoperability. Tasks: • Load and preprocess the Adult Income dataset. • Implement both Logistic Regression and Decision Trees. • Compare the models based on metrics such as accuracy, precision, recall, and F1-score. • Discuss the interoperability of both models and their suitability for the dataset.
11	Implement a Linear Support Vector Machine (SVM) to classify the Iris dataset. Visualize the decision boundary and discuss how the margin is determined. Tasks: • Load and preprocess the Iris dataset. • Implement a Linear SVM for binary classification (eg. classify Setosa vs Non-Setosa). • Visualize the decision boundary margin. • Discuss the concept of the margin and how it influences classification.
12	Implement and compare the performance of SVM classifiers with linear, polynomial, and RBF kernels on the Fashion MNIST dataset. Analyze the advantages and disadvantages of each kernel type. Tasks: • Load and preprocess the Fashion MNIST dataset. • Implement SVM with linear, polynomial and RBF kernels. • Compare the classification performance of each kernel. • Discuss the strength and weakness of each kernel type.

13	Implement and train Multilayer Feed-Forward Network (MLP) on the Wine Quality dataset. Experiment with different numbers of hidden layers and neurons and discuss how these choices affect the networks performance. Tasks: • Load and preprocess the Wine Quality dataset. • Design and Implement an MLP with varying architectures (different hidden layers and neurons). • Train and evaluate the network. • Discuss the impact of architecture choices on performance.
14	Implement and compare the performance of a neural network using different activation functions (Sigmoid, ReLu, Tanh) on the MNIST dataset. Analyze how each activation function affects the training process and classification accuracy. Tasks: • Load and preprocess the MNIST dataset. • Implement neural networks using Sigmoid, ReLu and Tanh activation functions. • Train and evaluate each network. Compare training times, convergence and classification accuracy.
15	Implement and perform hyper parameter tuning for a neural network on the Fashion MNIST dataset. Experiment with different learning rates, batch sizes, and epochs, and discuss the impact on model importance. Tasks: • Load and preprocess the Fashion MNIST dataset. • Experiment with different hyper parameters (learning rates, batch sizes, and epochs) . • Train and evaluate the network. • Discuss how hyperparameter choices affect model performance.
16	Implement and compare hierarchical (agglomerative) and partitional (K-means) clustering algorithms on the Mall Customer dataset. Discuss the strengths and weakness of each method based on clustering results and evaluation metrics. Tasks: • Load and preprocess the Mall Customer dataset. • Apply both hierarchical (agglomerative) and K-means clustering. • Compare results using metrics such as inertia silhouette score, and clustering visualization. • Discuss the advantages and disadvantages of each clustering method.
17	Implement and apply K-means clustering to the Digits dataset. Experiment with different number of clusters and evaluate the clustering results using metrics such as inertia and silhouette score. Analyze how the choice of K affects the clustering performance. Tasks: • Load and preprocess the Digits dataset. • Implement K-means clustering with various number of clusters. • Evaluate clustering performance using inertia and silhouette score. • Analyze the impact of the number of clusters on clustering quality.
18	Implement bootstrapping and cross validation on the Iris dataset. Compare the model performance metrics (eg. accuracy, F1-score) obtained using these resampling methods. Discuss the advantages and disadvantages of each method. Tasks: • Load and preprocess the Iris dataset. • Implement bootstrapping to generate multiple samples and evaluate the model. • Implement k-fold cross validation and evaluate the model. • Compare the performance metrics and discuss the pros and cons of each resampling method.

19	Implement bagging and boosting ensemble methods on the Titanic dataset. Compare the performance of both methods in terms of accuracy, precision, recall and F1-score. Discuss how each method improves model performance and their respective strength and weaknesses. Tasks: • Load and preprocess the Titanic dataset. • Implement bagging using a base classifier (eg. decision tree) and evaluate the performance. • Implement boosting using a boosting algorithm (eg. AdaBoost) and evaluate the performance. • Compare the performance metrics and discuss the strengths and weaknesses of each method.
20	Investigate the bias variance tradeoff using polynomial regression on the Boston Housing dataset. Plot the training and validation errors for various polynomial degrees and discuss the tradeoff of between bias and variance. Tasks: • Load and preprocess the Boston Housing dataset. • Implement polynomial regression with varying degrees. • Plot training and validation errors for each degree. • Discuss the bias variance tradeoff and its impact on model performance.

EXPERIMENT 1

Linear Regression with One Variable

Predicting Housing Prices using the California Housing Dataset

Course	Machine Learning (ML Lab)
Experiment	1 — Linear Regression (Single Variable)
Dataset	California Housing Dataset (sklearn)
Duration	3 Hours
Max. Marks	25 Marks
COs Covered	CO1, CO2, CO3, CO4

1. Aim

Implement linear regression with one variable on the California Housing dataset to predict housing prices based on a single feature (average number of rooms per dwelling — AveRooms). The experiment covers both gradient descent and the normal equation approaches, followed by performance evaluation and visualization.

2. Course Outcomes (COs) Addressed

CO	Description	Level	Bloom's Level
CO1	Understand the complexity of Machine Learning algorithms and their limitations.	K2	Understanding
CO2	Understand modern notions in data analysis-oriented computing.	K2	Understanding
CO3	Apply common Machine Learning algorithms in practice and implement their own.	K3	Applying
CO4	Perform experiments in Machine Learning using real-world data.	K3	Applying

3. Prerequisites

3.1 Theoretical Background

Before attending the lab, students must have a clear understanding of the following:

- Basics of supervised learning and regression problems
- Linear algebra: matrices, vectors, dot products, matrix inversion
- Calculus: partial derivatives, gradient of a function
- Cost functions: Mean Squared Error (MSE)
- Gradient Descent: batch gradient descent algorithm
- Normal Equation: closed-form solution for linear regression
- Python programming: NumPy, Pandas, Matplotlib, Scikit-learn basics

3.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

Complete the following tasks before arriving at the lab:

1. State the hypothesis function for linear regression with one variable.
2. Write the cost function $J(\theta_0, \theta_1)$ for linear regression and explain each term.
3. Derive the gradient descent update rule for θ_0 and θ_1.
4. State the Normal Equation formula and explain when it is preferred over gradient descent.

5. List two limitations of linear regression.
6. What does R-squared (R^2) measure? Write its formula.
7. Install required Python libraries: numpy, pandas, matplotlib, scikit-learn.

4. Theory

4.1 Linear Regression — Hypothesis

For a single feature x, the hypothesis is:

$$h\theta(x) = \theta_0 + \theta_1 x$$

where θ_0 is the bias (intercept) and θ_1 is the weight (slope).

4.2 Cost Function

The Mean Squared Error (MSE) cost function is minimized during training:

$$J(\theta_0, \theta_1) = (1/2m) \Sigma [h\theta(x^{-i}) - y^{-i}]^2$$

where m is the number of training examples, x^{-i} is the i-th input, and y^{-i} is the actual target.

4.3 Gradient Descent

Gradient Descent iteratively updates the parameters to minimize the cost function. The update rules are:

$$\theta_0 := \theta_0 - \alpha \cdot (1/m) \Sigma [h\theta(x^{-i}) - y^{-i}]$$

$$\theta_1 := \theta_1 - \alpha \cdot (1/m) \Sigma [h\theta(x^{-i}) - y^{-i}] \cdot x^{-i}$$

α is the learning rate (hyperparameter). Convergence is typically assessed by monitoring the cost function over iterations.

4.4 Normal Equation (Closed-Form Solution)

The Normal Equation computes the optimal parameters directly without iteration:

$$\theta = (X^T X)^{-1} X^T y$$

where X is the design matrix (with a column of ones prepended for the bias), y is the target vector, and θ is the parameter vector. This approach is suitable for smaller datasets but becomes computationally expensive ($O(n^3)$) for large feature sets.

4.5 Evaluation Metrics

Metric	Formula	Interpretation
MSE	$(1/m) \Sigma (\hat{y}_i - y_i)^2$	Lower is better; sensitive to outliers
RMSE	√MSE	Same unit as target variable; interpretable
R^2 Score	1 − SS_res / SS_tot	Range [0,1]; closer to 1 is better fit

5. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / Google Colab / VS Code
- Libraries: numpy, pandas, matplotlib, scikit-learn
- Computer with minimum 4 GB RAM

6. Step-by-Step Procedure

Step 1: Import Libraries and Load Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler

# Load dataset
data = fetch_california_housing(as_frame=True)
df = data.frame
print(df.head())
print(df.describe())
```

Step 2: Exploratory Data Analysis

```
# Examine the target variable distribution
plt.figure(figsize=(8, 4))
plt.hist(df['MedHouseVal'], bins=50, color='steelblue', edgecolor='white')
plt.title('Distribution of Median House Values')
plt.xlabel('Median House Value (in $100,000)')
plt.ylabel('Frequency')
plt.show()

# Scatter plot of chosen feature vs target
plt.scatter(df['AveRooms'], df['MedHouseVal'], alpha=0.3, s=5, color='steelblue')
plt.xlabel('Average Rooms per Dwelling (AveRooms)')
plt.ylabel('Median House Value')
plt.title('AveRooms vs MedHouseVal')
plt.show()
```

Step 3: Data Preprocessing

```
# Select single feature
X = df['AveRooms'].values
y = df['MedHouseVal'].values

# Clip outliers (optional, for better visualization)
mask = (X.flatten() < 10) & (y < 5)
X, y = X[mask], y[mask]

# Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)
```

Step 4: Linear Regression via Gradient Descent

```
def gradient_descent(X, y, lr=0.01, epochs=1000):
    m = len(y)
    theta0, theta1 = 0.0, 0.0
    cost_history = []
    for _ in range(epochs):
        y_pred = theta0 + theta1 * X.flatten()
        error  = y_pred - y
```

```
        theta0 -= lr * (1/m) * np.sum(error)
        theta1 -= lr * (1/m) * np.sum(error * X.flatten())
        cost_history.append((1/(2*m)) * np.sum(error**2))
    return theta0, theta1, cost_history

t0_gd, t1_gd, costs = gradient_descent(X_train_sc, y_train, lr=0.01, epochs=1000)
print(f'GD => theta0={t0_gd:.4f}, theta1={t1_gd:.4f}')

# Plot cost convergence
plt.plot(costs)
plt.title('Cost Function Convergence (Gradient Descent)')
plt.xlabel('Iterations')
plt.ylabel('MSE Cost')
plt.show()
```

Step 5: Linear Regression via Normal Equation

```
def normal_equation(X, y):
    # Add bias column of ones
    X_b = np.c_[np.ones(len(X)), X]
    theta = np.linalg.pinv(X_b.T @ X_b) @ X_b.T @ y
    return theta[0], theta[1]

t0_ne, t1_ne = normal_equation(X_train_sc, y_train)
print(f'NE => theta0={t0_ne:.4f}, theta1={t1_ne:.4f}')
```

Step 6: Evaluate Both Models

```
for name, t0, t1 in [('Gradient Descent', t0_gd, t1_gd),
                     ('Normal Equation',  t0_ne, t1_ne)]:
    y_pred = t0 + t1 * X_test_sc.flatten()
    mse  = mean_squared_error(y_test, y_pred)
    r2   = r2_score(y_test, y_pred)
    print(f'{name:20s}  MSE={mse:.4f}  RMSE={mse**0.5:.4f}  R2={r2:.4f}')
```

Step 7: Visualization — Fitted Regression Lines

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5), sharey=True)
for ax, (name, t0, t1) in zip(axes, [
        ('Gradient Descent', t0_gd, t1_gd),
        ('Normal Equation',  t0_ne, t1_ne)]):
    ax.scatter(X_test_sc, y_test, alpha=0.4, s=8, color='steelblue', label='Data')
    xs = np.linspace(X_test_sc.min(), X_test_sc.max(), 200)
    ax.plot(xs, t0 + t1*xs, color='crimson', lw=2, label='Fitted Line')
    ax.set_title(name)
    ax.set_xlabel('AveRooms (Scaled)')
    ax.set_ylabel('Median House Value')
    ax.legend()
plt.suptitle('Linear Regression: Gradient Descent vs Normal Equation')
plt.tight_layout()
plt.show()
```

7. Expected Results & Observations

Method	MSE (approx.)	RMSE (approx.)	R^2 (approx.)
Gradient Descent (lr=0.01, 1000 epochs)	0.55 – 0.70	0.74 – 0.84	0.15 – 0.30
Normal Equation (closed form)	0.55 – 0.70	0.74 – 0.84	0.15 – 0.30

Note: A single feature (AveRooms) has limited predictive power; R^2 values around 0.20–0.25 are expected. Both methods should yield nearly identical parameters and metrics.

8. Discussion Questions

- Why does gradient descent produce similar results to the normal equation when converged?
- What happens to the cost function if the learning rate is set too high or too low? Explain.
- Why is feature scaling necessary for gradient descent but not for the normal equation?
- How would you extend this experiment to multiple features (multivariate linear regression)?
- Explain why R^2 is low for this single-feature model. Which additional features might improve it?
- What is the time complexity of the normal equation? For what dataset size does gradient descent become preferable?

9. Conclusion (Template for Student Record)

In this experiment, linear regression was implemented using two methods:

- Gradient Descent with learning rate _____ and _____ epochs.
- Normal Equation using the closed-form formula.

The dataset used was the California Housing Dataset with the feature AveRooms. After preprocessing (train-test split and feature scaling), both models were trained and evaluated.

Method	MSE	RMSE	R^2
Gradient Descent			
Normal Equation			

It was observed that _____ [student fills their observations]. The experiment achieved the course outcomes CO1, CO2, CO3, and CO4 by implementing, analysing, and evaluating a fundamental ML algorithm on real-world data.

EXPERIMENT 2

Polynomial Regression

Predicting Miles Per Gallon (MPG) using the Auto MPG Dataset

Course	Machine Learning (ML Lab)
Experiment	3 — Polynomial Regression
Dataset	Auto MPG Dataset (UCI / sklearn-compatible CSV)
Duration	3 Hours
Max. Marks	25 Marks

1. Aim

Implement polynomial regression on the Auto MPG dataset to predict Miles Per Gallon (MPG) based on a single feature (engine displacement). The experiment covers fitting polynomial regression models of varying degrees, comparing them against a simple linear regression baseline using MSE and R^2, and visualizing the resulting curves against the data.

2. Prerequisites

3.1 Theoretical Background

Before attending the lab, students must have a clear understanding of the following:

- Basics of supervised learning and regression problems
- Linear regression with one variable (hypothesis, cost function)
- Concept of feature transformation / basis expansion
- Polynomial functions and degree of a polynomial
- Bias-variance trade-off: underfitting vs overfitting
- Cost functions: Mean Squared Error (MSE)
- Python programming: NumPy, Pandas, Matplotlib, Scikit-learn basics

3.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

Complete the following tasks before arriving at the lab:

- Write the hypothesis function for a degree-d polynomial regression model with one input feature.
- Explain how polynomial regression can be implemented as linear regression on transformed features.
- Define underfitting and overfitting in the context of polynomial degree selection.
- What does R-squared (R^2) measure? Write its formula.
- List one method to choose an appropriate polynomial degree (e.g., validation curve, cross-validation).
- State two risks of using a very high-degree polynomial on a small dataset.
- Install required Python libraries: numpy, pandas, matplotlib, scikit-learn.

3. Theory

4.1 Linear Regression — Baseline Hypothesis

For a single feature x, the linear hypothesis is:

$$\mathbf{h\theta(x) = \theta_0 + \theta_1 x}$$

This baseline model assumes a straight-line relationship between the feature and the target, which may underfit data with curved trends.

4.2 Polynomial Regression — Hypothesis

Polynomial regression extends linear regression by adding higher-order powers of the feature, while remaining linear in the parameters θ:

$$\mathbf{h\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \ldots + \theta^d x^d}$$

where d is the degree of the polynomial. The features x, x^2, …, x^d are treated as new independent variables, so the model can still be fit using ordinary least squares (linear regression machinery applied to expanded features).

4.3 Cost Function

As with linear regression, the Mean Squared Error (MSE) cost function is minimized during training:

$$J(\theta) = (1/2m)\ \Sigma\ [\ h\theta(x^{(i)}) - y^{(i)}\ ]^2$$

where m is the number of training examples, $x^{(i)}$ is the i-th input, and $y^{(i)}$ is the actual target.

4.4 Bias-Variance Trade-off and Degree Selection

Choosing the polynomial degree d controls model complexity:

- Low degree (e.g., d = 1): may underfit — high bias, fails to capture curvature in the data.
- Moderate degree (e.g., d = 2 or 3): often balances bias and variance well for smooth real-world trends.
- High degree (e.g., d ≥ 6): may overfit — low bias but high variance, fitting noise rather than the underlying trend, especially near the edges of the feature range.

The appropriate degree is typically selected by comparing training and test (or cross-validated) error across multiple degrees.

4.5 Evaluation Metrics

Metric	Formula	Interpretation
MSE	$(1/m)\ \Sigma\ (\hat{y}_i - y_i)^2$	Lower is better; sensitive to outliers
RMSE	√MSE	Same unit as target variable; interpretable
R^2 Score	1 − SS_res / SS_tot	Range [0,1]; closer to 1 is better fit

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / Google Colab / VS Code
- Libraries: numpy, pandas, matplotlib, scikit-learn
- Computer with minimum 4 GB RAM

5. Step-by-Step Procedure

Step 1: Import Libraries and Load Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score

# Load Auto MPG dataset (UCI repository, via seaborn's bundled copy or local CSV)
url = 'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/mpg.csv'
df = pd.read_csv(url)
print(df.head())
print(df.describe())
print(df.isna().sum())
```

Step 2: Exploratory Data Analysis

```
# Examine the target variable distribution
plt.figure(figsize=(8, 4))
plt.hist(df['mpg'], bins=30, color='steelblue', edgecolor='white')
plt.title('Distribution of MPG')
plt.xlabel('Miles Per Gallon (MPG)')
plt.ylabel('Frequency')
plt.show()

# Scatter plot of chosen feature vs target
plt.scatter(df['displacement'], df['mpg'], alpha=0.5, s=15, color='steelblue')
plt.xlabel('Engine Displacement (cu. in.)')
plt.ylabel('Miles Per Gallon (MPG)')
```

```
plt.title('Displacement vs MPG')
plt.show()
```

Step 3: Data Preprocessing

```
# Drop rows with missing values
df = df.dropna(subset=['displacement', 'mpg'])

# Select single feature and target
X = df[['displacement']].values
y = df['mpg'].values

# Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Step 4: Baseline — Simple Linear Regression

```
lin_model = LinearRegression()
lin_model.fit(X_train, y_train)

y_pred_lin = lin_model.predict(X_test)
mse_lin = mean_squared_error(y_test, y_pred_lin)
r2_lin  = r2_score(y_test, y_pred_lin)
print(f'Linear Regression  => MSE={mse_lin:.4f}  RMSE={mse_lin**0.5:.4f}  R2={r2_lin:.4f}')
```

Step 5: Polynomial Regression of Varying Degrees

```
degrees = [1, 2, 3, 4, 6, 8]
results = []
models  = {}

for d in degrees:
    poly = PolynomialFeatures(degree=d, include_bias=False)
    X_train_poly = poly.fit_transform(X_train)
    X_test_poly  = poly.transform(X_test)

    model = LinearRegression()
    model.fit(X_train_poly, y_train)

    y_pred = model.predict(X_test_poly)
    mse = mean_squared_error(y_test, y_pred)
    r2  = r2_score(y_test, y_pred)

    results.append((d, mse, mse**0.5, r2))
    models[d] = (poly, model)
    print(f'Degree {d:2d}  =>  MSE={mse:.4f}  RMSE={mse**0.5:.4f}  R2={r2:.4f}')
```

Step 6: Compare Polynomial Models with Linear Regression

```
results_df = pd.DataFrame(results, columns=['Degree', 'MSE', 'RMSE', 'R2'])
results_df.loc[-1] = ['Linear (deg=1 baseline)', mse_lin, mse_lin**0.5, r2_lin]
results_df = results_df.sort_index().reset_index(drop=True)
print(results_df)

plt.figure(figsize=(7, 4))
plt.plot(degrees, [r[3] for r in results], marker='o', color='crimson')
plt.xlabel('Polynomial Degree')
plt.ylabel('R² Score (Test Set)')
plt.title('Model Performance vs Polynomial Degree')
plt.grid(alpha=0.3)
plt.show()
```

Step 7: Visualization — Fitted Curves

```
xs = np.linspace(X.min(), X.max(), 300).reshape(-1, 1)

plt.figure(figsize=(9, 6))
plt.scatter(X_test, y_test, alpha=0.4, s=15, color='steelblue', label='Test Data')

# Linear baseline
plt.plot(xs, lin_model.predict(xs), color='black', lw=2, linestyle='--', label='Linear (deg=1)')

# A few representative polynomial degrees
```

```
for d, color in zip([2, 4, 8], ['crimson', 'green', 'orange']):
    poly, model = models[d]
    xs_poly = poly.transform(xs)
    plt.plot(xs, model.predict(xs_poly), color=color, lw=2, label=f'Degree {d}')

plt.xlabel('Engine Displacement (cu. in.)')
plt.ylabel('Miles Per Gallon (MPG)')
plt.title('Polynomial Regression Fits vs Linear Baseline')
plt.legend()
plt.tight_layout()
plt.show()
```

6. Expected Results & Observations

Model	MSE (approx.)	RMSE (approx.)	R^2 (approx.)
Linear Regression (degree 1)	18 – 22	4.2 – 4.7	0.60 – 0.66
Polynomial Regression (degree 2)	15 – 19	3.9 – 4.4	0.65 – 0.71
Polynomial Regression (degree 3–4)	14 – 18	3.7 – 4.2	0.66 – 0.72
Polynomial Regression (degree ≥ 8)	Varies – may rise on test set	Varies	Drops / fluctuates (overfitting)

Note: Engine displacement has a clear non-linear, slightly curved relationship with MPG, so low-degree polynomial models (degree 2–3) typically improve over the linear baseline. Very high degrees tend to overfit the training data and degrade test-set performance.

7. Discussion Questions

- Why can polynomial regression be fit using the same linear regression machinery (ordinary least squares)?
- At what polynomial degree does the model start to overfit the Auto MPG data? How can you detect this from the train vs test error?
- Why might a degree-2 or degree-3 polynomial outperform a simple linear model for this dataset?
- What is the effect of increasing polynomial degree on bias and variance?
- How would you use cross-validation to systematically select the best polynomial degree?
- What are the risks of using high-degree polynomials when extrapolating beyond the range of the training data?

8. Conclusion (Template for Student Record)

In this experiment, polynomial regression was implemented and compared against a simple linear regression baseline:

- Linear Regression as the baseline model (degree 1).
- Polynomial Regression models of degrees _____, _____, _____, and _____ .

The dataset used was the Auto MPG Dataset with the feature engine displacement, used to predict MPG. After preprocessing (handling missing values and train-test split), all models were trained and evaluated.

Model	MSE	RMSE	R^2
Linear Regression (deg 1)			
Polynomial (deg 2)			
Polynomial (deg 3)			
Polynomial (deg ___)			

It was observed that _____ [student fills their observations, including the best-performing degree and signs of overfitting/underfitting]. The experiment achieved the course outcomes CO1, CO2, CO3, and CO4 by implementing, analysing, and evaluating polynomial regression on real-world data.

EXPERIMENT 3

Ridge and Lasso Regression

Regularized Regression on the Diabetes Dataset

Course	Machine Learning (ML Lab)
Experiment	3 — Ridge & Lasso Regression
Dataset	Diabetes Dataset (sklearn)
Duration	3 Hours
Max. Marks	25 Marks

1. Aim

Implement Ridge and Lasso regression on the Diabetes dataset to predict disease progression using multiple features. The experiment covers regularized regression with hyperparameter tuning via cross-validation, followed by a performance comparison against standard (unregularized) linear regression using MSE and R^2.

2. Prerequisites

2.1 Theoretical Background

Before attending the lab, students must have a clear understanding of the following:

- Multivariate linear regression with multiple input features
- Concept of overfitting and model complexity in high-dimensional data
- L1 norm and L2 norm; the idea of penalizing large coefficients
- Regularization: how it trades off bias and variance
- Cross-validation: k-fold CV for hyperparameter selection
- Cost functions: Mean Squared Error (MSE)
- Python programming: NumPy, Pandas, Matplotlib, Scikit-learn basics

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

Complete the following tasks before arriving at the lab:

- Write the cost function for Ridge regression and identify the regularization term.
- Write the cost function for Lasso regression and identify the regularization term.
- Explain the key difference between L1 (Lasso) and L2 (Ridge) penalties in terms of their effect on coefficients.
- What does the hyperparameter α (alpha / lambda) control in Ridge and Lasso regression?
- Explain how k-fold cross-validation is used to select the best value of α.
- Why can Lasso regression be used for feature selection while Ridge cannot?
- Install required Python libraries: numpy, pandas, matplotlib, scikit-learn.

3. Theory

3.1 Standard Linear Regression — Baseline

For multiple features $x_1, x_2, \ldots, x_n$, the linear hypothesis is:

$$h\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_n x_n$$

Ordinary Least Squares (OLS) minimizes the MSE cost function with no penalty on the coefficients. With many correlated features, OLS coefficients can become large and unstable, leading to overfitting.

3.2 Ridge Regression (L2 Regularization)

Ridge regression adds an L2 penalty (sum of squared coefficients) to the cost function, shrinking coefficients toward zero without eliminating them:

$$J(\theta) = (1/2m)\ \Sigma\ [\ h\theta(x^{(i)}) - y^{(i)}\ ]^2 + \alpha\ \Sigma\ \theta_j^2$$

α (also called λ) is the regularization strength. Larger α shrinks coefficients more aggressively, reducing variance at the cost of some bias. Ridge keeps all features in the model but reduces their influence.

3.3 Lasso Regression (L1 Regularization)

Lasso regression adds an L1 penalty (sum of absolute coefficients) instead:

$$J(\theta) = (1/2m)\ \Sigma\ [\ h\theta(x^{(i)}) - y^{(i)}\ ]^2 + \alpha\ \Sigma\ |\theta_j|$$

Unlike Ridge, the L1 penalty can drive some coefficients exactly to zero, effectively performing automatic feature selection. This makes Lasso especially useful when only a subset of features is expected to be relevant.

3.4 Choosing the Regularization Strength via Cross-Validation

The hyperparameter α controls model complexity for both Ridge and Lasso:

- Very small α (→ 0): behaves like standard linear regression — may overfit if features are highly correlated or numerous.
- Moderate α: balances bias and variance, typically improving generalization on unseen data.
- Very large α: coefficients shrink toward zero (Ridge) or many become exactly zero (Lasso) — may underfit.

k-fold cross-validation (e.g., using RidgeCV / LassoCV or GridSearchCV) is used to evaluate multiple candidate α values on held-out folds and select the one with the lowest average validation error.

3.5 Evaluation Metrics

Metric	Formula	Interpretation
MSE	$(1/m)\ \Sigma\ (\hat{y}_i - y_i)^2$	Lower is better; sensitive to outliers
RMSE	$\sqrt{MSE}$	Same unit as target variable; interpretable
R^2 Score	$1 - SS_res / SS_tot$	Range [0,1]; closer to 1 is better fit

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / Google Colab / VS Code
- Libraries: numpy, pandas, matplotlib, scikit-learn
- Computer with minimum 4 GB RAM

5. Step-by-Step Procedure

Step 1: Import Libraries and Load Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso, RidgeCV, LassoCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
data = load_diabetes(as_frame=True)
df = data.frame
print(df.head())
print(df.describe())
print(df.info())
```

Step 2: Exploratory Data Analysis

```
# Examine the target variable distribution
plt.figure(figsize=(8, 4))
plt.hist(df['target'], bins=30, color='steelblue', edgecolor='white')
plt.title('Distribution of Disease Progression (Target)')
plt.xlabel('Target')
plt.ylabel('Frequency')
plt.show()
```

```
# Correlation of features with target
corr = df.corr()['target'].sort_values(ascending=False)
print(corr)

plt.figure(figsize=(8, 5))
plt.bar(corr.index[1:], corr.values[1:], color='steelblue')
plt.xticks(rotation=45)
plt.title('Feature Correlation with Target')
plt.tight_layout()
plt.show()
```

Step 3: Data Preprocessing

```
# Features and target (all 10 features used)
X = df.drop(columns=['target']).values
y = df['target'].values

# Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Feature scaling (important for Ridge/Lasso since penalty depends on coefficient scale)
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)
```

Step 4: Baseline — Standard Linear Regression

```
lin_model = LinearRegression()
lin_model.fit(X_train_sc, y_train)

y_pred_lin = lin_model.predict(X_test_sc)
mse_lin = mean_squared_error(y_test, y_pred_lin)
r2_lin  = r2_score(y_test, y_pred_lin)
print(f'Linear Regression  => MSE={mse_lin:.4f}  RMSE={mse_lin**0.5:.4f}  R2={r2_lin:.4f}')
print('Coefficients:', np.round(lin_model.coef_, 2))
```

Step 5: Ridge and Lasso Regression with Cross-Validation

```
alphas = np.logspace(-3, 2, 50)   # candidate alpha values: 0.001 to 100

# RidgeCV and LassoCV perform built-in k-fold cross-validation
ridge_cv = RidgeCV(alphas=alphas, cv=5)
ridge_cv.fit(X_train_sc, y_train)
print(f'Best alpha (Ridge): {ridge_cv.alpha_:.4f}')

lasso_cv = LassoCV(alphas=alphas, cv=5, max_iter=10000)
lasso_cv.fit(X_train_sc, y_train)
print(f'Best alpha (Lasso): {lasso_cv.alpha_:.4f}')

# Refit final models using the tuned alpha (RidgeCV/LassoCV already store the fitted estimator)
ridge_model = Ridge(alpha=ridge_cv.alpha_).fit(X_train_sc, y_train)
lasso_model = Lasso(alpha=lasso_cv.alpha_, max_iter=10000).fit(X_train_sc, y_train)
```

Step 6: Compare Performance Metrics

```
models = {
    'Linear Regression': lin_model,
    'Ridge Regression':  ridge_model,
    'Lasso Regression':  lasso_model,
}

results = []
for name, model in models.items():
    y_pred = model.predict(X_test_sc)
    mse = mean_squared_error(y_test, y_pred)
    r2  = r2_score(y_test, y_pred)
    results.append((name, mse, mse**0.5, r2))
    print(f'{name:20s}  MSE={mse:.4f}  RMSE={mse**0.5:.4f}  R2={r2:.4f}')

results_df = pd.DataFrame(results, columns=['Model', 'MSE', 'RMSE', 'R2'])
print(results_df)
```

Step 7: Visualization — Coefficients and Model Comparison

```
feature_names = df.drop(columns=['target']).columns

fig, ax = plt.subplots(figsize=(10, 5))
x_pos = np.arange(len(feature_names))
width = 0.25

ax.bar(x_pos - width, lin_model.coef_, width, label='Linear', color='steelblue')
ax.bar(x_pos,         ridge_model.coef_, width, label='Ridge', color='crimson')
ax.bar(x_pos + width, lasso_model.coef_, width, label='Lasso', color='green')

ax.set_xticks(x_pos)
ax.set_xticklabels(feature_names, rotation=45)
ax.set_ylabel('Coefficient Value')
ax.set_title('Coefficient Comparison: Linear vs Ridge vs Lasso')
ax.legend()
ax.axhline(0, color='black', lw=0.8)
plt.tight_layout()
plt.show()

# Bar chart of R2 scores
plt.figure(figsize=(6, 4))
plt.bar(results_df['Model'], results_df['R2'], color=['steelblue', 'crimson', 'green'])
plt.ylabel('R² Score (Test Set)')
plt.title('Model Performance Comparison')
plt.show()
```

6. Expected Results & Observations

Model	MSE (approx.)	RMSE (approx.)	R^2 (approx.)
Linear Regression	2900 – 3100	54 – 56	0.45 – 0.50
Ridge Regression (tuned α)	2850 – 3050	53 – 55	0.46 – 0.51
Lasso Regression (tuned α)	2850 – 3050	53 – 55	0.46 – 0.51

Note: On the Diabetes dataset (10 features, low multicollinearity), Ridge and Lasso typically yield modest improvements over standard linear regression, with Lasso also shrinking some coefficients (e.g., for weakly correlated features) to exactly zero. Exact values depend on the random train-test split and the chosen alpha grid.

7. Discussion Questions

- Why is feature scaling important before applying Ridge or Lasso regression?
- Which features did Lasso shrink to exactly zero? What does this imply about their importance?
- How does the value of α selected by cross-validation affect the bias-variance trade-off?
- Under what conditions would you expect Ridge to outperform Lasso, and vice versa?
- Why might Ridge and Lasso perform only marginally better than standard linear regression on this dataset?
- How would Elastic Net (which combines L1 and L2 penalties) be useful compared to Ridge or Lasso alone?

8. Conclusion (Template for Student Record)

In this experiment, Ridge and Lasso regression were implemented and compared against a standard linear regression baseline:

- Linear Regression as the unregularized baseline model.
- Ridge Regression with cross-validated α = _____ .
- Lasso Regression with cross-validated α = _____ .

The dataset used was the Diabetes Dataset with all 10 available features, used to predict disease progression. After preprocessing (train-test split and feature scaling), all models were trained, tuned via cross-validation, and evaluated.

Model	MSE	RMSE	R^2
Linear Regression			
Ridge Regression			
Lasso Regression			

It was observed that _____ [student fills their observations, including the best-performing model and which features Lasso eliminated]. The experiment achieved the intended outcomes by implementing, tuning, and evaluating regularized regression models on real-world data.

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MACHINE LEARNING LABORATORY
EXPERIMENT 4
Logistic Regression for Classification

Diagnosis of Breast Cancer using the Scikit-Learn Breast Cancer Dataset

Course	Machine Learning (ML Lab)
Experiment	4 — Logistic Regression (Classification)
Dataset	Breast Cancer Wisconsin Dataset (sklearn.datasets.load_breast_cancer)
Duration	3 Hours
Max. Marks	25 Marks

1. Aim

Implement a Logistic Regression model from scratch using Gradient Descent and compare it with the Scikit-Learn implementation on the Breast Cancer dataset. Evaluate the classification model using metrics such as Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix.

2. Prerequisites

2.1 Theoretical Background

- Difference between Regression and Classification tasks.
- The Sigmoid (Logistic) activation function and mapping outputs to probabilities.
- Binary Cross-Entropy Loss (Log Loss) function.
- Gradient Descent optimization for updating logistic weights.
- Evaluation metrics: True Positives (TP), False Positives (FP), True Negatives (TN), False Negatives (FN).

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

- Why can standard linear regression not be used reliably for classification problems?
- Write the mathematical equation for the Sigmoid function and state its domain and range.
- Write the Binary Cross-Entropy (Log Loss) cost function used in Logistic Regression.
- Derive or state the gradient update rule for weights in Logistic Regression.
- Define Precision and Recall. In a breast cancer diagnostic tool, which metric is more critical to optimize, and why?
- What is a Confusion Matrix? Draw a 2x2 structural layout of it.

3. Theory

3.1 Hypothesis Function

Logistic regression maps real-valued linear predictions to probabilities between 0 and 1 using the Sigmoid function:
h_theta(x) = g(theta^T X) = 1 / (1 + e^(-theta^T X))
A decision boundary (typically 0.5) is applied to classify instances into binary targets (0 or 1).

3.2 Cost Function (Log Loss)

Because the standard MSE loss creates a non-convex function when combined with the sigmoid activation, Logistic Regression utilizes the Log Loss / Binary Cross-Entropy function:
J(theta) = -1/m * sum(y^(i) * log(h_theta(x^(i))) + (1 - y^(i)) * log(1 - h_theta(x^(i))))

3.3 Gradient Descent Updates

The parameter update rules closely resemble linear regression but use the logistic hypothesis function:
theta_j := theta_j - alpha * (1/m) * sum((h_theta(x^(i)) - y^(i)) * x_j^(i))

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / Google Colab / VS Code
- Libraries: numpy, pandas, matplotlib, scikit-learn
- Computer with minimum 4 GB RAM

5. Step-by-Step Procedure

Step 1: Import Libraries and Load Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

# Load dataset
cancer = load_breast_cancer(as_frame=True)
df = cancer.frame
print(df.head())
print(df.info())
```

Step 2: Data Preprocessing and Splitting

```
X = cancer.data.values
y = cancer.target.values

# Split into Train and Test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature Scaling
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
```

Step 3: Logistic Regression from Scratch using Gradient Descent

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def fit_logistic_regression(X, y, lr=0.1, epochs=1000):
    m, n = X.shape
    X_b = np.c_[np.ones(m), X]
    weights = np.zeros(n + 1)

    for _ in range(epochs):
        z = np.dot(X_b, weights)
        y_pred = sigmoid(z)
        error = y_pred - y
        gradient = np.dot(X_b.T, error) / m
        weights -= lr * gradient
    return weights

# Train Scratch Model
weights = fit_logistic_regression(X_train_sc, y_train, lr=0.1, epochs=1000)
```

Step 4: Scratch Model Inference & Evaluation

```
def predict(X, weights):
    X_b = np.c_[np.ones(X.shape[0]), X]
    probabilities = sigmoid(np.dot(X_b, weights))
```

```
        return [1 if p >= 0.5 else 0 for p in probabilities]

    y_pred_scratch = predict(X_test_sc, weights)
    print(f'Scratch Accuracy: {accuracy_score(y_test, y_pred_scratch):.4f}')
```

Step 5: Scikit-Learn Logistic Regression

```
    sklearn_model = LogisticRegression(max_iter=1000)
    sklearn_model.fit(X_train_sc, y_train)
    y_pred_sklearn = sklearn_model.predict(X_test_sc)
    print(f'Sklearn Accuracy: {accuracy_score(y_test, y_pred_sklearn):.4f}')
```

Step 6: Comparative Metrics & Confusion Matrix

```
    def print_metrics(y_true, y_pred, name):
        print(f'\n--- Performance of {name} ---')
        print(f'Accuracy:  {accuracy_score(y_true, y_pred):.4f}')
        print(f'Precision: {precision_score(y_true, y_pred):.4f}')
        print(f'Recall:    {recall_score(y_true, y_pred):.4f}')
        print(f'F1 Score:  {f1_score(y_true, y_pred):.4f}')
        print(f'Confusion Matrix:\n{confusion_matrix(y_true, y_pred)}')

    print_metrics(y_test, y_pred_scratch, 'Scratch Implementation')
    print_metrics(y_test, y_pred_sklearn, 'Scikit-Learn Implementation')
```

6. Expected Results & Observations

Method	Accuracy (approx.)	Precision (approx.)	Recall (approx.)	F1-Score (approx.)
Scratch Model (GD)	0.94 – 0.97	0.95 – 0.98	0.94 – 0.98	0.95 – 0.98
Scikit-Learn Model	0.96 – 0.98	0.96 – 0.99	0.96 – 0.99	0.96 – 0.99

7. Discussion Questions

1. Why does standard Logistic Regression utilize Log Loss instead of Mean Squared Error (MSE)? Explain with respect to convexity.
2. What is the impact of learning rate hyperparameter alpha on the loss convergence graph?
3. In a diagnostic environment where missing a cancer diagnosis (False Negative) is highly critical, which metric should be maximized, and why?
4. How does feature scaling influence gradient descent performance during your manual scratch implementation loop?

8. Conclusion (Template for Student Record)

In this experiment, a Logistic Regression model was successfully built from scratch using gradient descent rules and systematically compared with the industrial standard Scikit-Learn package. Real-world validation was executed over the Breast Cancer Wisconsin Dataset.

Method	Accuracy	Precision	Recall	F1 Score
Scratch Model (GD)				
Scikit-Learn Model				

EXPERIMENT 5
MLE and MAP Estimations in Logistic Regression

Probabilistic Parameter Estimation and Regularization Effects on Breast Cancer Data

Course	Machine Learning (ML Lab)
Experiment	5 — Parameter Estimation: MLE vs MAP in Logistic Regression
Dataset	Breast Cancer Wisconsin Dataset (sklearn.datasets.load_breast_cancer)
Duration	3 Hours
Max. Marks	25 Marks

1. Aim

Estimate the parameters of a Logistic Regression model using Maximum Likelihood Estimation (MLE) and Maximum A Posteriori (MAP) estimation on the Breast Cancer Wisconsin dataset. Compare the resulting parameter estimates, evaluate classification performance metrics, and analyze the structural effects of L1 (Laplace) and L2 (Gaussian) regularization priors.

2. Prerequisites

2.1 Theoretical Background

- Frequentist vs. Bayesian parameter estimation frameworks.
- The concept of likelihood functions for Bernoulli distributions.
- Prior distributions: Gaussian priors (L2 regularization) vs. Laplace priors (L1 regularization).
- Bayes' Theorem applied to parameter spaces.
- Log-likelihood transformations for numerical stability.

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

- Define Maximum Likelihood Estimation (MLE) and write down the likelihood formula for a binary dataset under Logistic Regression.
- State Bayes' Theorem as it pertains to parameter estimation, highlighting the Likelihood, Prior, and Posterior components.
- Mathematically show how adding a zero-mean Gaussian prior P(w) to the parameter space relates to L2 regularization.
- Mathematically show how adding a zero-mean Laplace prior P(w) to the parameter space relates to L1 regularization.
- What structural difference occurs in the final weight vectors when swapping an L2 prior for an L1 prior?
- Why does pure MLE tend to yield exceptionally large parameter values if data is linearly separable or features are multi-collinear?

3. Theory

3.1 Maximum Likelihood Estimation (MLE)

In a frequentist perspective, MLE assumes parameters (w) are fixed but unknown. It optimizes parameters by maximizing the probability of observing the given training data labels:
L(w) = prod(P(y^(i) | x^(i); w)) = prod([h_w(x^(i))]^(y^(i)) * [1 - h_w(x^(i))]^(1 - y^(i)))
Maximizing this likelihood is mathematically equivalent to minimizing the unregularized Log-Loss objective function.

3.2 Maximum A Posteriori (MAP) Estimation

In a Bayesian perspective, parameters are treated as random variables possessing an initial prior distribution P(w). MAP maximizes the posterior distribution:
P(w | X, y) proportional to P(y | X, w) * P(w)
Taking the negative log transforms the product into a sum:
Objective_MAP = - log P(y | X, w) - log P(w) = LogLoss(w) - log P(w)
• Gaussian Prior: P(w) = N(0, sigma^2) leads to a squared penalty term (alpha * sum(w_j^2)), matching L2 regularization.
• Laplace Prior: P(w) = Laplace(0, b) leads to an absolute penalty term (beta * sum(|w_j|)), matching L1 regularization.

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / VS Code
- Libraries: numpy, pandas, matplotlib, scikit-learn
- Computer with minimum 4 GB RAM

5. Step-by-Step Procedure

Step 1: Import Libraries and Load Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, log_loss

# Load the dataset
cancer = load_breast_cancer(as_frame=True)
X = cancer.data.values
y = cancer.target.values
features = cancer.feature_names

# Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature scaling (Critical for regularization scaling parity)
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
```

Step 2: Implement MLE Model (No Regularization)

```
# In scikit-learn, penalty=None approximates pure MLE
mle_model = LogisticRegression(penalty=None, solver='lbfgs', max_iter=10000)
mle_model.fit(X_train_sc, y_train)

w_mle = mle_model.coef_[0]
y_pred_mle = mle_model.predict(X_test_sc)
print(f'MLE (Unregularized) Test Accuracy: {accuracy_score(y_test, y_pred_mle):.4f}')
print(f'MLE Norm (sum of squared weights): {np.sum(w_mle**2):.4f}')
```

Step 3: Implement MAP Model with L2 Regularization (Gaussian Prior)

```
# penalty='l2' represents a Gaussian prior over the weights
map_l2_model = LogisticRegression(penalty='l2', C=0.1, solver='lbfgs', max_iter=10000)
map_l2_model.fit(X_train_sc, y_train)

w_l2 = map_l2_model.coef_[0]
y_pred_l2 = map_l2_model.predict(X_test_sc)
print(f'MAP L2 (Gaussian Prior) Test Accuracy: {accuracy_score(y_test, y_pred_l2):.4f}')
print(f'MAP L2 Norm (sum of squared weights): {np.sum(w_l2**2):.4f}')
```

Step 4: Implement MAP Model with L1 Regularization (Laplace Prior)

```
# penalty='l1' represents a Laplace prior
map_l1_model = LogisticRegression(penalty='l1', C=0.1, solver='saga', max_iter=10000,
random_state=42)
map_l1_model.fit(X_train_sc, y_train)

w_l1 = map_l1_model.coef_[0]
y_pred_l1 = map_l1_model.predict(X_test_sc)
print(f'MAP L1 (Laplace Prior) Test Accuracy: {accuracy_score(y_test, y_pred_l1):.4f}')
print(f'MAP L1 Sparsity (Zero Coefficients): {np.sum(w_l1 == 0)} out of {len(w_l1)}')
```

Step 5: Tabulate and Plot Parameter Profiles

```
df_weights = pd.DataFrame({
    'Feature': features,
    'MLE (No Prior)': w_mle,
    'MAP L2 (Gaussian Prior)': w_l2,
    'MAP L1 (Laplace Prior)': w_l1
})
print('\nWeight Comparison Table:\n', df_weights.head(10))

# Visualization
plt.figure(figsize=(12, 6))
plt.plot(features[:15], w_mle[:15], label='MLE (No Prior)', marker='o', color='red',
linestyle='--')
plt.plot(features[:15], w_l2[:15], label='MAP L2 (Gaussian)', marker='s', color='blue')
plt.plot(features[:15], w_l1[:15], label='MAP L1 (Laplace)', marker='^', color='green')
plt.axhline(0, color='black', linestyle=':', alpha=0.5)
plt.xticks(rotation=45, ha='right')
plt.title('Parameter Estimates Comparison (First 15 Features)')
plt.ylabel('Weight Value')
plt.legend()
plt.tight_layout()
plt.show()
```

6. Expected Results & Observations

Estimation Framework	Corresponding Prior	Magnitude of Weights (Norm)	Sparsity (Zero Weights)	Expected Test Accuracy
MLE	None (Uniform Prior)	Very Large (> 50.0)	0 Features	93.0% – 95.5%
MAP (L2)	Gaussian Prior	Small / Shrunk (< 5.0)	0 Features	96.5% – 98.2%
MAP (L1)	Laplace Prior	Intermediate	Many (Automated Selection)	95.5% – 97.5%

7. Discussion Questions

- Why does MLE result in significantly larger magnitude values for parameters compared to MAP techniques?
- How does the variance parameter of a Gaussian prior influence the regularized value (alpha/C) inside scikit-learn?
- Which estimation layout would you deploy if your target system operates on low-power hardware requiring strict memory reductions? Justify using your observed L1 sparsity counts.
- Explain how a Bayesian prior acts as a mathematical buffer against overfitting when working with small datasets containing many parameters.

8. Conclusion (Template for Student Record)

In this experiment, the parameter space of a binary classification model was quantified through both frequentist (MLE) and Bayesian (MAP) estimation schemes over the Breast Cancer Wisconsin dataset.

Framework	**Accuracy Score**	**Sum of Squared Weights**	**Total Zeroed Features**
Maximum Likelihood (MLE)			
MAP L2 (Gaussian Prior)			
MAP L1 (Laplace Prior)			

It was observed that MAP estimation with a Laplace prior produced exactly _____ zeroed weight parameters, while a Gaussian prior limited parameter values to a range of _____. This confirms that regularization priors effectively govern generalization performance. This laboratory exercise satisfies milestones through explicit probabilistic model formulation and hands-on validation.

EXPERIMENT 6
Logistic Regression with Feature Scaling Analysis
Disease Likelihood Prediction using the Pima Indians Diabetes Dataset

Course	Machine Learning (ML Lab)
Experiment	6 — Effect of Feature Scaling on Logistic Regression
Dataset	Pima Indians Diabetes Dataset (diabetes.csv)
Duration	3 Hours
Max. Marks	25 Marks

1. Aim
Implement a Logistic Regression model to predict the likelihood of diabetes using the Pima Indians Diabetes dataset. Evaluate and compare the model's convergence behavior, parameter weights, and comprehensive classification performance metrics (Accuracy, Precision, Recall, F1-Score) when trained with versus without Feature Scaling (Standardization).

2. Prerequisites

2.1 Theoretical Background
- Logistic mapping via Sigmoid function for probability classification distributions.
- Optimization sensitivity: How disparate feature magnitudes affect the geometry of Gradient Descent contours.
- The mathematical process of Standardization (Z-score normalization).
- Interpretation of multi-metric evaluation parameters in clinically unbalanced sets.

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)
- Write the explicit formula for Z-score normalization (Standardization), defining every symbol used.
- If one feature has a range of [0, 1] (e.g., Pedigree Function) and another has a range of [0, 200] (e.g., Glucose), how does this difference structurally affect the gradient updates of their respective weights (w_j)?
- Define the mathematical equations for Precision and Recall in binary classification.
- What is geometric ill-conditioning in optimization, and why does feature scaling mitigate it?
- Does feature scaling alter the raw correlation coefficients between variables? Why or why not?

3. Theory

3.1 The Impact of Feature Scales on Gradient Descent
When input features have vastly different ranges, the cost function contours become highly elongated, elliptical shapes. Consequently, gradient descent optimization steps will oscillate wildly in directions with large scales, requiring a drastically lower learning rate or significantly more iterations to converge.
Standardization transforms individual feature vectors to have zero mean and unit variance:
x_scaled = (x - mean) / std_dev
This scales the contours into symmetric spherical topologies, allowing gradient descent vectors to point directly toward the global minimum, guaranteeing smoother and accelerated convergence profile updates.

4. Software & Hardware Requirements
- Python 3.8 or above
- Jupyter Notebook / VS Code
- Libraries: numpy, pandas, matplotlib, scikit-learn

- Pima Indians Diabetes Dataset (.csv format)

5. Step-by-Step Procedure

Step 1: Load and Preprocess the Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

# Load dataset from URL or local path
url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-
diabetes.data.csv'
columns = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI',
'DiabetesPedigree', 'Age', 'Outcome']
df = pd.read_csv(url, names=columns)

X = df.drop('Outcome', axis=1).values
y = df['Outcome'].values

# Split into Train and Test splits (70/30 layout for stable variance observation)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Step 2: Train Logistic Regression Without Feature Scaling

```
# Fit model with a fixed solver constraint to examine unscaled gradient environments
model_unscaled = LogisticRegression(solver='saga', max_iter=1000, random_state=42)
model_unscaled.fit(X_train, y_train)

y_pred_unscaled = model_unscaled.predict(X_test)
```

Step 3: Train Logistic Regression With Feature Scaling

```
# Apply Standardization pipeline
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model_scaled = LogisticRegression(solver='saga', max_iter=1000, random_state=42)
model_scaled.fit(X_train_scaled, y_train)

y_pred_scaled = model_scaled.predict(X_test_scaled)
```

Step 4: Performance Generation Matrix

```
def calculate_metrics(y_true, y_pred, label):
    return {
        'Framework': label,
        'Accuracy': accuracy_score(y_true, y_pred),
        'Precision': precision_score(y_true, y_pred),
        'Recall': recall_score(y_true, y_pred),
        'F1-Score': f1_score(y_true, y_pred)
    }

res_unscaled = calculate_metrics(y_test, y_pred_unscaled, 'Without Scaling')
res_scaled = calculate_metrics(y_test, y_pred_scaled, 'With Scaling (Standardized)')

metrics_df = pd.DataFrame([res_unscaled, res_scaled])
print(metrics_df.to_string(index=False))
```

Step 5: Visualize Coefficient Magnitude Discrepancies

```
plt.figure(figsize=(10, 5))
plt.bar(columns[:-1], model_unscaled.coef_[0], alpha=0.6, label='Unscaled Weights', color='red')
plt.bar(columns[:-1], model_scaled.coef_[0], alpha=0.6, label='Scaled Weights', color='blue')
plt.axhline(0, color='black', linestyle=':')
plt.xticks(rotation=30)
plt.ylabel('Weight Parameter Coefficient Value')
```

```
plt.title('Weight Coefficients: Unscaled vs. Standardized Features')
plt.legend()
plt.tight_layout()
plt.show()
```

6. Expected Results & Observations

Condition Framework	Accuracy (approx.)	Precision (approx.)	Recall (approx.)	Optimization Behavior
Without Scaling	0.64 – 0.70	0.55 – 0.65	0.30 – 0.45	Slow convergence or solver fails to fully minimize Log-Loss.
With Scaling (Z-score)	0.74 – 0.78	0.68 – 0.75	0.60 – 0.68	Fast, stable minimization; clear, reliable feature weight tracking.

7. Discussion Questions

5. Why does the 'SAGA' or 'Gradient Descent' optimizer converge poorly when trained on unscaled features like Insulin and BloodPressure?
6. When features are scaled, are their relative coefficient values more reflective of their actual statistical importance to the model? Explain.
7. In a diabetes screening test, what is the risk associated with a low Recall score compared to a low Precision score?
8. Does feature scaling change the position of the optimal separating hyperplane, or does it simply alter the numeric mechanics of finding it?

8. Conclusion (Template for Student Record)

In this experiment, a Logistic Regression pipeline was developed to predict diabetes likelihood using the Pima Indians Diabetes dataset. The model was trained and thoroughly evaluated under both unscaled and standard-scaled conditions.

Experiment Environment	Accuracy	Precision	Recall	F1 Score
Without Feature Scaling				
With Feature Scaling (Standardized)				

EXPERIMENT 7
Text Classification using Naïve Bayes Variants
Comparative Analysis of Multinomial and Bernoulli Models on 20 Newsgroups Data

Course	Machine Learning (ML Lab)
Experiment	7 — Naïve Bayes Text Classification Analysis
Dataset	20 Newsgroups Dataset (sklearn.datasets.fetch_20newsgroups)
Duration	3 Hours
Max. Marks	25 Marks

1. Aim
Implement a Naïve Bayes document classifier using the 20 Newsgroups text dataset. Perform feature extraction and data preprocessing, implement and evaluate Multinomial Naïve Bayes and Bernoulli Naïve Bayes, and compare their performance using macro-averaged Accuracy and F1-Score while discussing the architectural strengths and weaknesses of each variant.

2. Prerequisites

2.1 Theoretical Background
- Bayes' Theorem for conditional probability calculation and posterior optimization.
- The Conditional Independence Assumption in Naïve Bayes frameworks.
- Bag-of-Words text modeling representations: Word Frequency counts vs. Binary Presence indicators.
- Handling data sparsity: The mathematical principle of Laplace Smoothing (Additive Smoothing).

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)
- State Bayes' Theorem mathematically for calculating the probability of a document class C_k given a text feature vector x.
- Explain the foundational difference between Count Vectorization (used for Multinomial NB) and Binary Vectorization (used for Bernoulli NB).
- What is the mathematical purpose of Laplace Smoothing, and what value does the hyperparameter α traditionally take to avoid zero probabilities?
- Why does the Naïve Bayes classifier apply log-transformations to its conditional probability products during internal computational cycles?
- How does a vocabulary containing highly correlated words violate the core structural assumptions of a Naïve Bayes model?

3. Theory

3.1 Multinomial vs. Bernoulli Representation Architecture
Naïve Bayes algorithms leverage the conditional independence assumption, meaning the occurrence of a given feature is assumed independent of all other features given the target class variable.
• Multinomial Naïve Bayes: Models feature vectors representing word frequencies (how many times word x_i appears inside a document). It is highly suitable for multi-page documents where word count distributions dictate categorical topics.
• Bernoulli Naïve Bayes: Models boolean/binary feature vectors (whether word x_i is present or absent inside a text, discarding frequency counts). It explicitly penalizes the non-occurrence of words, making it ideal for short texts or spam detection where specific trigger keywords define classification.

4. Software & Hardware Requirements
- Python 3.8 or above

- Jupyter Notebook / VS Code
- Libraries: numpy, pandas, matplotlib, scikit-learn
- Internet connection to download the 20 Newsgroups dataset via scikit-learn

5. Step-by-Step Procedure

Step 1: Load and Filter the 20 Newsgroups Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression # Reference context
from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.metrics import accuracy_score, f1_score, classification_report

# Select 4 distinct structural topics to optimize training speeds
categories = ['sci.med', 'sci.space', 'comp.graphics', 'talk.politics.misc']
train_data = fetch_20newsgroups(subset='train', categories=categories, remove=('headers',
'footers', 'quotes'))
test_data = fetch_20newsgroups(subset='test', categories=categories, remove=('headers',
'footers', 'quotes'))

X_train, y_train = train_data.data, train_data.target
X_test, y_test = test_data.data, test_data.target
```

Step 2: Preprocess and Transform via Text Vectorization Pipelines

```
# Configuration 1: Word Count Frequency Vectorization (for Multinomial NB)
vectorizer_multinomial = CountVectorizer(stop_words='english', min_df=2)
X_train_counts = vectorizer_multinomial.fit_transform(X_train)
X_test_counts = vectorizer_multinomial.transform(X_test)

# Configuration 2: Binary Presence Vectorization (for Bernoulli NB)
vectorizer_bernoulli = CountVectorizer(stop_words='english', binary=True, min_df=2)
X_train_binary = vectorizer_bernoulli.fit_transform(X_train)
X_test_binary = vectorizer_bernoulli.transform(X_test)
```

Step 3: Implement and Train Multinomial Naïve Bayes

```
# Alpha=1.0 instantiates standard Laplace additive smoothing
clf_multinomial = MultinomialNB(alpha=1.0)
clf_multinomial.fit(X_counts_train if 'X_counts_train' in locals() else X_train_counts, y_train)
y_pred_multi = clf_multinomial.predict(X_test_counts)

print('--- Multinomial Naïve Bayes Complete ---')
print(f'Accuracy: {accuracy_score(y_test, y_pred_multi):.4f}')
```

Step 4: Implement and Train Bernoulli Naïve Bayes

```
clf_bernoulli = BernoulliNB(alpha=1.0)
clf_bernoulli.fit(X_train_binary, y_train)
y_pred_bern = clf_bernoulli.predict(X_test_binary)

print('--- Bernoulli Naïve Bayes Complete ---')
print(f'Accuracy: {accuracy_score(y_test, y_pred_bern):.4f}')
```

Step 5: Tabulate Performance Profile Matrices

```
metrics_summary = {
    'Model Framework': ['Multinomial Naïve Bayes (Frequency)', 'Bernoulli Naïve Bayes
(Binary)'],
    'Test Accuracy': [accuracy_score(y_test, y_pred_multi), accuracy_score(y_test,
y_pred_bern)],
    'Macro F1-Score': [f1_score(y_test, y_pred_multi, average='macro'), f1_score(y_test,
y_pred_bern, average='macro')]
}
df_res = pd.DataFrame(metrics_summary)
print('\nPerformance Comparisons:\n', df_res.to_string(index=False))

# Quick bar chart visual comparison
```

```
df_res.set_index('Model Framework').plot(kind='bar', figsize=(8, 5))
plt.ylabel('Score Metric Scale')
plt.title('Naïve Bayes Variant Analysis on Text Corpora')
plt.xticks(rotation=0)
plt.ylim(0.5, 1.0)
plt.tight_layout()
plt.show()
```

6. Expected Results & Observations

Model Variant	Expected Accuracy	Expected F1-Score	Algorithmic Behavior Summary
Multinomial NB	0.78 – 0.86	0.77 – 0.85	Succeeds on text categories because word usage frequency maps strong signals to topics.
Bernoulli NB	0.68 – 0.76	0.65 – 0.74	Underperforms compared to Multinomial on long texts due to ignoring explicit term repetition profiles.

6.2 Structural Strengths and Weaknesses Comparison Matrix

Naïve Bayes Variant	Core Architectural Strengths	Core Architectural Weaknesses
Multinomial Naïve Bayes	Takes word recurrence weights into account; optimal for standard length papers, emails, articles.	Completely drops context, grammar rules, word order (Bag-of-Words constraints); sensitive to document lengths if unnormalized.
Bernoulli Naïve Bayes	Robust framework against noise words; explicitly accounts for the absence of negative indicators; highly effective on short messages (SMS / Twitter streams).	Completely ignores frequency counts; an active keyword occurring 10 times carries the exact statistical weight as occurring once.

7. Discussion Questions

- Why does Multinomial Naïve Bayes generally outperform Bernoulli Naïve Bayes on standard-length articles from the 20 Newsgroups corpus?
- Under what specific circumstances or constraints would you deploy a Bernoulli Naïve Bayes text classifier over a Multinomial structure?
- Explain how Laplace smoothing alters parameter equations if a target test token was never observed inside the training dictionary setup.
- If the conditional independence assumption is mathematically violated in natural text structures, why do Naïve Bayes algorithms retain strong ranking and classification capacities in production environments?

8. Conclusion (Template for Student Record)

In this laboratory experiment, a comparative analysis was executed between Multinomial and Bernoulli Naïve Bayes architectures over selected classes of the 20 Newsgroups dataset.

Naïve Bayes Framework Variant	**Final Test Accuracy Score**	**Final Macro F1-Score**
Multinomial Naïve Bayes (Counts Vectorized)		
Bernoulli Naïve Bayes (Binary Vectorized)		

EXPERIMENT 8
Image Classification with K-Nearest Neighbors (KNN)
Hyperparameter Tuning (K-Value Analysis) on the Fashion MNIST Dataset

Course	Machine Learning (ML Lab)
Experiment	8 — K-Nearest Neighbors (Image Classification)
Dataset	Fashion MNIST Dataset (keras.datasets.fashion_mnist / sklearn equivalent)
Duration	3 Hours
Max. Marks	25 Marks

1. Aim
Implement the K-Nearest Neighbors (KNN) algorithm to perform multi-class image classification using the Fashion MNIST dataset. Experiment with different values of the hyperparameter K (K = 1, 3, 5, 11) to evaluate their direct impact on classification accuracy and execution efficiency during training and inference.

2. Prerequisites

2.1 Theoretical Background
- Instance-based, non-parametric, lazy learning paradigms.
- Distance metrics for high-dimensional numeric vectors (Minkowski, Euclidean, and Manhattan distances).
- The structural mechanics of majority voting and handling tied boundaries.
- Image feature matrices: Normalizing 2D pixel structures (28 times 28) into flat arrays (784 features).
- The Curse of Dimensionality and its computational implications on search latency.

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)
- Why is the K-Nearest Neighbors (KNN) algorithm categorized as a 'lazy learner' or an 'instance-based algorithm'?
- Write down the mathematical formula for the Euclidean distance between two flattened image vectors, U and V, of length N.
- What is the mathematical and operational effect of setting an exceptionally low value for K (e.g., $K=1$) versus an exceptionally high value (e.g., $K=100$) on the decision boundaries?
- Why are raw pixel intensities traditionally scaled from the range $[0, 255]$ down to $[0, 1]$ before computing distance vectors?
- Explain how the test phase computational complexity of KNN scales relative to the number of instances (M) in the training matrix.

3. Theory

3.1 Core K-Nearest Neighbors Logic
KNN classifies a novel test sample by finding the K closest data points in the training matrix based on a distance metric, such as Euclidean distance:
d(x, q) = sqrt(sum_{i=1}^{n} (x_i - q_i)^2)
Once the K closest instances are computed, the label is determined via a majority vote among those neighbors.
• Hyperparameter K Impact: Small values of K create highly complex decision boundaries that capture fine patterns but are susceptible to noise (overfitting). Large values smooth out boundaries, capturing global trends but potentially erasing minority class boundaries (underfitting).
• Computational Efficiency: Because KNN stores all historical arrays and computes distances across every sample on each inquiry, its search cost scales as $O(M imes N)$ per query, presenting latency bottlenecks on massive datasets like image sets.

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / VS Code
- Libraries: numpy, pandas, matplotlib, scikit-learn, tensorflow/keras (for downloading dataset only)
- Minimum 8 GB RAM recommended

5. Step-by-Step Procedure

Step 1: Fetch and Preprocess the Fashion MNIST Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time
from tensorflow.keras.datasets import fashion_mnist
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix

# Load dataset
(X_train_raw, y_train), (X_test_raw, y_test) = fashion_mnist.load_data()

# Subsample to optimize execution speeds within the lab time window
train_mask = np.random.choice(X_train_raw.shape[0], 5000, replace=False)
test_mask = np.random.choice(X_test_raw.shape[0], 500, replace=False)

X_train_sub, y_train = X_train_raw[train_mask], y_train[train_mask]
X_test_sub, y_test = X_test_raw[test_mask], y_test[test_mask]

# Flatten image grids (28x28) into continuous arrays (784 features)
X_train = X_train_sub.reshape(X_train_sub.shape[0], -1)
X_test = X_test_sub.reshape(X_test_sub.shape[0], -1)

# Scale intensities from [0, 255] down to [0, 1]
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Drop', 'Coat', 'Sandal', 'Shirt',
'Sneaker', 'Bag', 'Ankle boot']
print(f'Training feature array matrix shape: {X_train.shape}')
```

Step 2: Experiment with Varying K Values

```
k_values = [1, 3, 5, 11]
results = []

for k in k_values:
    print(f'Training KNN framework with K={k}...')
    knn = KNeighborsClassifier(n_neighbors=k, metric='euclidean')

    # Benchmarking inference processing latencies
    start_time = time.time()
    knn.fit(X_train, y_train)
    train_time = time.time() - start_time

    start_time = time.time()
    y_pred = knn.predict(X_test)
    test_time = time.time() - start_time

    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average='macro')

    results.append({
        'K-Value': k,
        'Accuracy': acc,
        'Macro F1-Score': f1,
        'Inference Latency (s)': test_time
    })
```

```
results_df = pd.DataFrame(results)
print('\nHyperparameter Performance Summary Table:\n')
print(results_df.to_string(index=False))
```

Step 3: Plot Accuracy and Inference Latencies Profiles

```
fig, ax1 = plt.subplots(figsize=(8, 4))

color = 'tab:blue'
ax1.set_xlabel('Hyperparameter Value (K)')
ax1.set_ylabel('Test Accuracy Score', color=color)
ax1.plot(results_df['K-Value'], results_df['Accuracy'], marker='o', color=color, linewidth=2)
ax1.tick_params(axis='y', labelcolor=color)

ax2 = ax1.twinx()
color = 'tab:red'
ax2.set_ylabel('Inference Delay Time (Seconds)', color=color)
ax2.plot(results_df['K-Value'], results_df['Inference Latency (s)'], marker='s', color=color,
linestyle='--')
ax2.tick_params(axis='y', labelcolor=color)

plt.title('KNN Parameter Grid Search Analysis: Accuracy vs. Efficiency')
fig.tight_layout()
plt.show()
```

6. Expected Results & Observations

K Neighbor Value	Expected Accuracy Range	Computational Efficiency Impact	Underlying Structural Bias Context
K = 1	78.0% – 82.0%	Extremely high query lookups, minimal search bounds.	Low bias, high variance. Sensitive to overlapping outliers inside image manifolds.
K = 3	81.0% – 84.5%	High distance matrix sort loops.	Optimal boundary balance for localized image patterns.
K = 5	81.5% – 0.85%	Slightly increased search times.	Stabilized majority voting baseline; minimal noise fluctuations.
K = 11	79.0% – 83.0%	Maximized search calculations.	Increased bias, low variance. Over-smooths decision areas, misclassifying minority boundary items.

7. Discussion Questions

- Why does the KNN training phase complete instantaneously ($O(1)$) while the testing/inference phase is extremely demanding computationally?
- How does changing the value of K from 1 to 11 structurally alter the bias-variance trade-off of an instance-based model?
- Why does raw Euclidean distance often degrade when processing uncompressed images with large feature spaces (e.g., $O(784)$ dimensions)? Discuss the Curse of Dimensionality.
- What strategies can be implemented to accelerate the inference search overhead of a KNN classifier when using massive reference databases?

8. Conclusion (Template for Student Record)

In this experiment, a multi-class instance-based K-Nearest Neighbors classifier was deployed over localized blocks of the Fashion MNIST image corpus to analyze the trade-offs between hyperparameter values, generalization accuracy, and computational lookup bottlenecks.

Tested K Parameter	Observed Accuracy Score	Observed Macro F1-Score	Measured Inference Delay (s)
K = 1			
K = 3			
K = 5			
K = 11			

Based on experimental findings, setting K = ___ yielded the highest test accuracy of ___%, while the highest compute lookup delay occurred when K reached ___. It is verified that instance-based models require balanced parameter optimization to prevent noise-induced overfitting while remaining within viable run-time execution limits.

EXPERIMENT 9
Customer Segmentation Using ID3 Decision Trees

Tree Structure Visualization and Feature Importance Profiling on Online Retail Purchasing Data

Course	Machine Learning (ML Lab)
Experiment	9 — Decision Tree Classifier (ID3 Algorithm)
Dataset	Online Retail Behavioral Dataset (UCI Machine Learning Repository / Synthetic Behavioral Blocks)
Duration	3 Hours
Max. Marks	25 Marks

1. Aim

Implement a Decision Tree classifier based on the ID3 (Iterative Dichotomiser 3) algorithm to segment customers into behavioral categories using the Online Retail dataset. Extract and interpret Information Gain formulas, generate visual tree splits, quantify relative feature importances, and discuss how structural recursive rules aid in understanding commercial purchasing behavior.

2. Prerequisites

2.1 Theoretical Background

- Supervised tree-structured classification models (Root nodes, Internal decision splits, Leaf nodes).
- Information theory metrics: Shannon Entropy as a metric for categorical impurity.
- Information Gain criteria as a feature splitting selector metric.
- RFM modeling metrics for retail behavioral processing: Recency, Frequency, and Monetary parameters.
- The multi-way split characteristics of ID3 and its sensitivity to high-cardinality discrete variables.

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

- Write the complete mathematical expression for Shannon Entropy $H(S)$ of a dataset sample S containing C unique categorical labels.
- Define the formula for Information Gain $Gain(S, A)$ achieved by splitting dataset S along feature attribute A.
- Explain why standard ID3 Decision Trees cannot directly ingest continuous quantitative columns without an intermediary discretization or multi-bin boundary processing routine.
- Define what an RFM framework signifies within retail data preprocessing, explicitly detailing the behavioral value of 'Recency', 'Frequency', and 'Monetary Value'.
- Why does the basic ID3 algorithm demonstrate structural bias favoring attributes containing highly unique categorical levels (high cardinality), and how does this affect tree generalization bounds?

3.1 ID3 Architecture and Information Theory Concepts

The ID3 algorithm builds decision trees top-down through a greedy recursive strategy. At each structural level, it determines the variable that minimizes class entropy (or maximizes the empirical Information Gain):
$H(S) = - \sum_{i=1}^{c} p_i \log_2(p_i)$
$Gain(S, A) = H(S) - \sum_{v \in Values(A)} (|S_v| / |S|) * H(S_v)$
• Analyzing Customer Behavior via Trees: Decision trees yield white-box models. Every branch from root to terminal leaf establishes a clear combination of business logic paths (e.g., 'If Recency <= 30 days AND Monetary Value = High, Then Class = Loyal Customer'). This transparency allows marketing and product teams to easily isolate and profile behavioral subgroups.
• Feature Importance Profile: Features appearing at the topmost tiers of the tree drive the most substantial reduction in categorical impurity, indicating their dominant predictive value across the customer population.

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / VS Code Workspace
- Libraries: numpy, pandas, scikit-learn, matplotlib
- Graphviz library installed system-wide (for export_graphviz rendering capabilities)

5. Step-by-Step Procedure

Step 1: Load and Preprocess the Retail Dataset into RFM Features

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Create a robust synthetic sample representing the Online Retail data structure
np.random.seed(42)
n_samples = 800

data = {
    'Recency_Tier': np.random.choice(['Recent', 'Dormant', 'At-Risk'], n_samples, p=[0.5, 0.3,
0.2]),
    'Frequency_Tier': np.random.choice(['Low', 'Medium', 'High'], n_samples, p=[0.4, 0.4, 0.2]),
    'Monetary_Tier': np.random.choice(['Low', 'Medium', 'High'], n_samples, p=[0.3, 0.5, 0.2]),
    'Segment': np.random.choice(['Bronze', 'Silver', 'Gold'], n_samples, p=[0.4, 0.4, 0.2])
}
df = pd.DataFrame(data)

# Map discrete string categoricals into encoded integers for algorithm compatibility
for col in ['Recency_Tier', 'Frequency_Tier', 'Monetary_Tier']:
    df[col] = df[col].astype('category').cat.codes

# Define map for targets
target_map = {'Bronze': 0, 'Silver': 1, 'Gold': 2}
df['Segment'] = df['Segment'].map(target_map)

X = df.drop('Segment', axis=1)
y = df['Segment']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
print(df.head())
```

Step 2: Train the Decision Tree Model Using Entropy Criteria (ID3 Mode)

```
# Sklearn uses an optimized CART layout; enforcing criterion='entropy' mimics the splitting
choice logic of ID3
# max_depth limits tree growth to ensure clear interpretability during evaluation stages
clf_tree = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
clf_tree.fit(X_train, y_train)

y_pred = clf_tree.predict(X_test)
print(f'Model Classification Accuracy: {accuracy_score(y_test, y_pred):.4f}')
```

Step 3: Structural Tree Visualization and Feature Importance Analysis

```
# Visualize tree structure path arrays
plt.figure(figsize=(14, 8))
plot_tree(clf_tree, feature_names=list(X.columns), class_names=['Bronze', 'Silver', 'Gold'],
filled=True, rounded=True)
plt.title('Customer Segmentation Decision Tree Structure (Entropy/ID3 Split)')
plt.tight_layout()
plt.show()

# Extract, sort, and display computed Gini/Entropy Feature Importances
importances = clf_tree.feature_importances_
feat_imp = pd.Series(importances, index=X.columns).sort_values(ascending=False)
```

```
plt.figure(figsize=(8, 4))
feat_imp.plot(kind='bar', color='teal')
plt.ylabel('Relative Importance Scaling')
plt.title('RFM Feature Importance Breakdown')
plt.tight_layout()
plt.show()
print('\nFeature Importance Metrics:\n', feat_imp)
```

6. Expected Results & Observations

Extracted Feature	Relative Importance Value	Observed Tree Placement	Empirical Behavioral Signal Analysis
Monetary_Tier	0.45 – 0.60	Root Node / High Tier	Acts as the strongest driver for splitting high-value segments (Gold) from standard tiers.
Frequency_Tier	0.25 – 0.35	Level 1 Internal Split Nodes	Differentiates steady recurring transactions from sporadic purchasing events.
Recency_Tier	0.10 – 0.20	Lower Decision Nodes / Leaves	Refines boundaries by isolating dormant records requiring re-engagement campaigns.

7. Discussion Questions

- Explain how a visual path from root to leaf in this decision tree directly maps to an actionable, rule-based business strategy.
- Why does the ID3 splitting criteria struggle when handling columns with high numeric uniqueness (e.g., raw 'Total Invoice Value' values), and how does Gain Ratio mitigate this limitation?
- If a decision tree runs with an unrestricted depth parameter, what happens to the training vs. testing accuracy profile? Explain the concept of tree overfitting.
- What specific structural indicators from the tree plot would guide a marketing manager to launch a retention campaign targeting 'at-risk' high-value accounts?

8. Conclusion (Template for Student Record)

In this laboratory experiment, a white-box ID3 Decision Tree framework was constructed over retail purchasing profiles. The model parsed RFM fields to discover meaningful behavior branches and segment profiles.

Behavioral Feature Column Name	Observed Feature Importance % Score	Observed Tree Placement Tier
Monetary Value Tier		
Frequency Tier		
Recency Tier		

MACHINE LEARNING LABORATORY
EXPERIMENT 10
Model Comparison: Logistic Regression vs. Decision Trees

Income Level Prediction and Interpretability Profiling on the Adult Income Dataset

Course	Machine Learning (ML Lab)
Experiment	10 — Comparative Classification & Interpretability Analysis
Dataset	Adult Income Dataset (UCI Machine Learning Repository)
Duration	3 Hours
Max. Marks	25 Marks

1. Aim

Implement and systematically compare a parametric model (Logistic Regression) and a non-parametric model (Decision Tree Classifier) on the Adult Income dataset to predict individual income levels (>50K vs. <=50K). Evaluate both frameworks across standard classification matrices (Accuracy, Precision, Recall, F1-Score) and critically analyze their architectural interpretability and domain suitability.

2. Prerequisites

2.1 Theoretical Background

- Parametric vs. non-parametric model classification boundaries.
- Logistic Regression mathematical mapping using the Sigmoid link function and log-odds coefficients.
- Decision Tree recursive splitting via information criterion boundaries (Gini Index / Entropy).
- One-Hot Encoding methods for high-cardinality discrete nominal variables.
- Interpretability taxonomies: Global structural interpretability vs. localized parameter tracking weights.

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

9. Differentiate between parametric and non-parametric algorithms regarding their underlying statistical assumptions about feature distributions.
10. What is the physical interpretation of an exponentiated weight coefficient (e^w_j) in a binary Logistic Regression framework?
11. Why must multi-categorical features (such as 'workclass' or 'education') be encoded using One-Hot techniques before feeding into Logistic Regression, whereas a Decision Tree can conceptually process nominal categories directly?
12. Define the term 'Interpretability' or 'Model Interoperability' and explain the trade-offs between a model's complexity and its transparency.
13. If a dataset exhibits a severe class imbalance (e.g., 75% earning <=50K and 25% earning >50K), why is classification Accuracy considered a misleading performance indicator compared to macro F1-Score?

3. Theory

3.1 Parametric vs. Non-Parametric Modeling Foundations

Supervised classification frameworks are generally bifurcated into parametric and non-parametric architectures. Logistic Regression is a foundational parametric algorithm that assumes a functional form for the decision boundary. Specifically, it models the log-odds of the target variable as a linear combination of the input features:
log(p / (1 - p)) = w_0 + w_1*x_1 + w_2*x_2 + ... + w_n*x_n
By passing this linear combination through the Sigmoid function, the model maps any real-valued feature vector into a probability value between 0 and 1. This generates a smooth, linear decision hyperplane in the feature space. In contrast, a Decision Tree is a non-parametric model that makes no structural distributions assumptions. Instead, it recursively partitions the feature space into orthogonal hyper-rectangles based on statistical purity metrics like Gini

Impurity or Information Gain (Entropy). This allows Decision Trees to natively capture complex non-linear combinations and step functions without requiring manual feature engineering.

3.2 Multi-Categorical Data Transformation Mechanics

Real-world feature repositories like the Adult Income dataset contain extensive nominal categorical variables (such as 'workclass', 'education', and 'occupation'). Parametric algorithms require inputs to be numerical vectors to calculate weight gradients. Therefore, high-cardinality discrete classes must be converted using One-Hot Encoding into a series of sparse binary dummy variables. For a category with 'K' unique classes, 'K-1' dummy features are generated to prevent multi-collinearity (the dummy variable trap). Conversely, non-parametric tree models process nominal columns natively by finding the optimal binary multi-categorical split subsets, bypassing the strict necessity of geometric scaling or feature expansion.

3.3 Model Interpretability and Transparency Taxonomy

Model interoperability is defined as the level to which a human can consistently comprehend the reasoning behind a model's operational decisions. Logistic Regression features high global parametric interpretability: the magnitude and sign of the exponentiated coefficients (e^w_j) represent the exact change in odds ratio per unit movement of that specific feature while holding other factors constant. This gives clear, stable insights but lacks the ability to map deep feature interaction paths. On the other hand, a Decision Tree provides exceptional visual and rule-based interpretability at shallow depths (e.g., max_depth <= 5) by explicitly tracing a series of boolean conditions. However, if left completely unpruned, a Decision Tree will quickly grow highly irregular and overfit, causing its visual transparency to degrade into an opaque, unstable structure.

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / VS Code
- Libraries: numpy, pandas, matplotlib, scikit-learn
- Adult Income Dataset (adult.data from UCI Machine Learning Repository)

5. Step-by-Step Procedure

Step 1: Load and Preprocess the Adult Income Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Load dataset directly from URL resource
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data'
columns = ['age', 'workclass', 'fnlwgt', 'education', 'education-num', 'marital-status',
           'occupation', 'relationship', 'race', 'sex', 'gain', 'loss', 'hours', 'country',
'income']
df = pd.read_csv(url, names=columns, skipinitialspace=True, na_values='?')

# Drop rows containing missing tracking fields to maintain a baseline standard
df = doc_df = df.dropna()

# Encode target variable: income >50K maps to 1, otherwise 0
df['income'] = df['income'].apply(lambda x: 1 if '>50K' in x else 0)

# Select structural numerical and nominal columns to test processing boundaries
cat_cols = ['workclass', 'education', 'marital-status', 'occupation', 'relationship', 'sex']
num_cols = ['age', 'education-num', 'gain', 'loss', 'hours']

# Apply One-Hot dummy transformation across selected categoricals
df_encoded = pd.get_dummies(df[cat_cols + num_cols + ['income']], columns=cat_cols,
drop_first=True)

X = df_encoded.drop('income', axis=1).values
```

```
y = df_encoded['income'].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Apply feature scaling required for parametric linear optimization matrices
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print(f'Encoded dataset feature bounds: {X_train_scaled.shape}')
```

Step 2: Train and Test Both Competitive Framework Pipelines

```
# Configuration A: Parametric Logistic Regression Pipeline
log_reg = LogisticRegression(max_iter=1000, random_state=42)
log_reg.fit(X_train_scaled, y_train)
y_pred_lr = log_reg.predict(X_test_scaled)

# Configuration B: Non-Parametric Decision Tree Pipeline
tree_clf = DecisionTreeClassifier(max_depth=5, criterion='entropy', random_state=42)
tree_clf.fit(X_train, y_train) # Ingests unscaled arrays safely
y_pred_dt = tree_clf.predict(X_test)
```

Step 3: Tabulate and Compare Metric Matrices

```
def compile_metrics(y_true, y_pred, name):
    return {
         'Model Framework': name,
         'Accuracy': accuracy_score(y_true, y_pred),
         'Precision': precision_score(y_true, y_pred),
         'Recall': recall_score(y_true, y_pred),
         'F1-Score': f1_score(y_true, y_pred)
    }

summary_data = [
    compile_metrics(y_test, y_pred_lr, 'Logistic Regression'),
    compile_metrics(y_test, y_pred_dt, 'Decision Tree Classifier')
]

metrics_df = pd.DataFrame(summary_data)
print('\nPerformance Comparisons Matrix:\n', metrics_df.to_string(index=False))
```

6. Expected Results & Observations

Model Classifier	Expected Accuracy	Expected Precision	Expected Recall	Expected F1-Score
Logistic Regression	0.82 – 0.85	0.70 – 0.74	0.55 – 0.62	0.62 – 0.67
Decision Tree (Depth=5)	0.83 – 0.86	0.74 – 0.78	0.50 – 0.58	0.60 – 0.65

6.2 Interpretability and Structural Comparison Matrix

Interpretability Attribute	Logistic Regression Paradigm	Decision Tree Paradigm
Global Transparency Mechanics	Highly stable. Generates an explicit algebraic line equations formula. Explains how features individually weigh against the target log-odds vector.	Highly visual for constrained depths. Maps clean hierarchical flow diagrams easily read as logical branches.
Handling Feature Interactions	Weak capability. Assumes completely independent, additive feature relationships unless interaction tokens are manually engineered.	Strong capability. Naturally captures interactive feature sequences implicitly through recursive sub-node paths.

Sensitivity to Outliers / Scaling	Highly sensitive. Requires precise standardization scaling routines to preserve coefficient weights and gradient convergence.	Completely invariant. Monotonic transformations or isolated outlying spikes do not skew single-variable thresholds.

7. Discussion Questions

- How do One-Hot encoding steps expand feature spaces, and how does this affect Logistic Regression vs. Decision Trees?
- If a bank uses your model to approve credits, why might regulators prefer a shallow Decision Tree or Logistic Regression over a Deep Neural Network? Discuss interpretability constraints.
- Explain how an unpruned Decision Tree creates jagged, overfitted decision boundaries compared to the smooth linear plane generated by a Logistic Regression model.
- Which model is better suited for the Adult Income dataset when deploying a pipeline intended for real-time inference on missing or noisy fields? Why?

8. Conclusion (Template for Student Record)

In this laboratory experiment, a comparative evaluation was executed between a Logistic Regression classifier and a Decision Tree model on the Adult Income dataset. The models were evaluated on performance metrics and interpretability trade-offs.

Tested Classifier Model Architecture	**Observed Test Accuracy**	**Observed Precision**	**Observed Recall**	**Observed F1-Score**
Logistic Regression (Parametric Regularized)				
Decision Tree Classifier (Non-Parametric)				

EXPERIMENT 11

Support Vector Machines

Linear SVM Classification and Margin Visualization using the Iris Dataset

Course	Machine Learning (ML Lab)
Experiment	11 — Support Vector Machines (Linear SVM)
Dataset	Iris Dataset (UCI / sklearn-compatible CSV)
Duration	3 Hours
Max. Marks	25 Marks

1. Aim

Implement a Linear Support Vector Machine (SVM) to classify the Iris dataset, treated as a binary classification problem (Setosa vs Non-Setosa). The experiment covers loading and preprocessing the data, training a linear SVM classifier, visualizing the resulting decision boundary and margin, and discussing how the margin is determined and how it influences classification.

2. Prerequisites

2.1 Theoretical Background

Before attending the lab, students must have a clear understanding of the following:

- Basics of supervised learning and binary classification problems
- Concept of a separating hyperplane in feature space
- Support vectors and the notion of margin
- Margin maximization as an optimization problem
- Soft-margin SVM and the role of the regularization parameter C
- Confusion matrix, accuracy, precision, recall, and F1-score
- Python programming: NumPy, Pandas, Matplotlib, Scikit-learn basics

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

Complete the following tasks before arriving at the lab:

- Write the equation of a separating hyperplane in terms of the weight vector w and bias b.
- Define the geometric margin of a linear classifier and explain why SVM seeks to maximize it.
- Define support vectors and explain why only they determine the position of the decision boundary.
- Write the soft-margin SVM optimization objective and explain the role of slack variables.
- Explain the effect of the regularization parameter C on the width of the margin and on misclassification tolerance.
- State one reason feature scaling is important before training an SVM.
- Install required Python libraries: numpy, pandas, matplotlib, scikit-learn.

3. Theory

3.1 Linear SVM — Maximum Margin Classifier

For linearly separable data with labels $y \in \{-1, +1\}$, a linear SVM seeks a hyperplane that separates the two classes while maximizing the margin — the distance between the hyperplane and the nearest data points of each class. The hyperplane is defined as:

$$\mathbf{w^T x + b = 0}$$

Points are classified using the sign of $w^T x + b$. The decision boundary is chosen to satisfy $y_i(w^T x_i + b) \geq 1$ for every training point, so that the margin, equal to $2 / ||w||$, is as large as possible.

3.2 Support Vectors

Support vectors are the training points that lie exactly on the margin boundaries (where $y_i(w^T x_i + b) = 1$). These points alone determine the position and orientation of the separating hyperplane; all other points could be removed from the training set without changing the decision boundary, which is why SVM is described as a sparse, margin-based classifier.

3.3 Soft-Margin SVM

Real datasets are rarely perfectly separable, so a slack variable $\xi_i \geq 0$ is introduced for each point to allow some margin violations. The optimization objective becomes:

$$\textbf{minimize } (1/2)||w||^2 + C \Sigma \xi_i \text{ , subject to } y_i(w^T x_i + b) \geq 1 - \xi_i$$

The regularization parameter C controls the trade-off between maximizing the margin and minimizing classification error on the training set. A small C allows a wider margin with more tolerance for misclassified points (higher bias, lower variance), while a large C penalizes margin violations heavily, producing a narrower margin that fits the training data more closely (lower bias, higher variance).

3.4 Decision Function

Once trained, a new point x is classified using:

$$f(x) = \text{sign}(w^T x + b)$$

The magnitude $|w^T x + b|$ reflects how far a point lies from the decision boundary, which is why SVM decision boundaries are often visualized together with the two parallel margin lines $w^T x + b = +1$ and $w^T x + b = -1$.

3.5 Evaluation Metrics

Metric	Formula	Interpretation
Accuracy	(TP + TN) / Total	Overall fraction of correct predictions
Precision	TP / (TP + FP)	Of predicted positives, fraction actually positive
Recall	TP / (TP + FN)	Of actual positives, fraction correctly identified
F1-Score	2 · (Precision · Recall) / (Precision + Recall)	Harmonic mean of precision and recall

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / Google Colab / VS Code
- Libraries: numpy, pandas, matplotlib, scikit-learn
- Computer with minimum 4 GB RAM

5. Step-by-Step Procedure

Step 1: Import Libraries and Load Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# Load Iris dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target_names[iris.target]
print(df.head())
print(df['species'].value_counts())
```

Step 2: Exploratory Data Analysis

```
# Use two features for easy 2D visualization: petal length and petal width
```

```
plt.figure(figsize=(7, 5))
for species, color in zip(iris.target_names, ['crimson', 'steelblue', 'green']):
    subset = df[df['species'] == species]
    plt.scatter(subset['petal length (cm)'], subset['petal width (cm)'],
                label=species, alpha=0.7, color=color)
plt.xlabel('Petal Length (cm)')
plt.ylabel('Petal Width (cm)')
plt.title('Petal Length vs Petal Width by Species')
plt.legend()
plt.show()
```

Step 3: Create Binary Labels and Select Features

```
# Binary target: Setosa (1) vs Non-Setosa (0)
y_binary = (iris.target == 0).astype(int)

# Use petal length and petal width — Setosa is linearly separable on these features
X = df[['petal length (cm)', 'petal width (cm)']].values
```

Step 4: Data Preprocessing

```
# Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y_binary, test_size=0.2, random_state=42, stratify=y_binary)

# Feature scaling (important for SVM, which is distance/margin based)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Step 5: Train the Linear SVM

```
svm_model = SVC(kernel='linear', C=1.0)
svm_model.fit(X_train_scaled, y_train)

y_pred = svm_model.predict(X_test_scaled)
```

Step 6: Evaluate the Model

```
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print(f'Accuracy={acc:.4f}  Precision={prec:.4f}  Recall={rec:.4f}  F1={f1:.4f}')
print('Confusion Matrix:')
print(cm)
```

Step 7: Visualize the Decision Boundary and Margin

```
def plot_svm_boundary(model, X, y, scaler):
    X_orig = scaler.inverse_transform(X)
    plt.figure(figsize=(8, 6))
    plt.scatter(X_orig[y == 1, 0], X_orig[y == 1, 1], color='crimson', label='Setosa')
    plt.scatter(X_orig[y == 0, 0], X_orig[y == 0, 1], color='steelblue', label='Non-Setosa')

    ax = plt.gca()
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()

    xx = np.linspace(xlim[0], xlim[1], 200)
    yy = np.linspace(ylim[0], ylim[1], 200)
    YY, XX = np.meshgrid(yy, xx)
    xy_scaled = scaler.transform(np.c_[XX.ravel(), YY.ravel()])
    Z = model.decision_function(xy_scaled).reshape(XX.shape)

    # Decision boundary (0) and margins (+1, -1)
    ax.contour(XX, YY, Z, colors='black',
```

```
                levels=[-1, 0, 1], alpha=0.7,
                linestyles=['--', '-', '--'])

    # Highlight support vectors
    sv_orig = scaler.inverse_transform(model.support_vectors_)
    ax.scatter(sv_orig[:, 0], sv_orig[:, 1], s=120,
                facecolors='none', edgecolors='black', linewidths=1.5,
                label='Support Vectors')

    plt.xlabel('Petal Length (cm)')
    plt.ylabel('Petal Width (cm)')
    plt.title('Linear SVM Decision Boundary and Margin (Setosa vs Non-Setosa)')
    plt.legend()
    plt.show()

plot_svm_boundary(svm_model, X_train_scaled, y_train, scaler)
```

6. Expected Results & Observations

Metric	Expected Value (approx.)	Notes
Accuracy	1.00	Setosa is linearly and very clearly separable from the other two species
Precision	1.00	No Non-Setosa points are expected to be misclassified as Setosa
Recall	1.00	All Setosa points are expected to be correctly identified
Margin width	Relatively wide	Petal length/width for Setosa forms a tight, well-separated cluster

Note: Because Setosa is almost perfectly linearly separable from Versicolor and Virginica on petal length and petal width, the linear SVM is expected to achieve (or come very close to) perfect classification, with a clearly visible, wide margin and only a small number of support vectors near the boundary.

7. Discussion Questions

- Why does maximizing the margin, rather than simply finding any separating hyperplane, tend to produce a classifier that generalizes well?
- Which points in the training set became support vectors, and what do they have in common?
- How would the margin and decision boundary change if the regularization parameter C were made very small? Very large?
- Why is feature scaling important before training an SVM, and what would happen to the margin computation without it?
- Would a linear SVM be sufficient to separate Versicolor from Virginica using the same two features? Why or why not?
- How does the concept of margin relate to the model's robustness to small perturbations in the input data?

8. Conclusion (Template for Student Record)

In this experiment, a Linear Support Vector Machine was implemented to classify the Iris dataset as a binary problem (Setosa vs Non-Setosa):

- Features used: petal length and petal width.
- Regularization parameter used: C = _____ .
- Number of support vectors identified: _____ .

After preprocessing (feature scaling and train-test split), the model was trained and evaluated using accuracy, precision, recall, and F1-score, and the decision boundary together with the margin was visualized.

Metric	Value
Accuracy	
Precision	
Recall	
F1-Score	

It was observed that _____ [student fills their observations, including how wide the margin was, which points became support vectors, and how the margin relates to the model's generalization ability]. The experiment achieved the course outcomes CO1, CO2, CO3, and CO4 by implementing, analysing, and evaluating a linear SVM classifier on real-world data.

EXPERIMENT 12

SVM Kernel Comparison

Comparing Linear, Polynomial, and RBF Kernels on the Fashion MNIST Dataset

Course	Machine Learning (ML Lab)
Experiment	12 — SVM Kernel Comparison (Linear, Polynomial, RBF)
Dataset	Fashion MNIST Dataset (Zalando Research / keras & sklearn-compatible)
Duration	3 Hours
Max. Marks	25 Marks

Department of Computer Science & Engineering

1. Aim

Implement and compare Support Vector Machine (SVM) classifiers using linear, polynomial, and RBF kernels on the Fashion MNIST dataset. The experiment covers loading and preprocessing the image data, training SVMs with each kernel type, comparing their classification performance (accuracy, precision, recall, F1-score, and training time), and analyzing the strengths and weaknesses of each kernel.

2. Prerequisites

2.1 Theoretical Background

Before attending the lab, students must have a clear understanding of the following:

- Basics of supervised learning and multi-class classification problems
- SVM fundamentals: hyperplanes, margin, and support vectors
- The kernel trick and the idea of mapping data to a higher-dimensional feature space
- Linear, polynomial, and RBF (Gaussian) kernel functions
- One-vs-Rest / One-vs-One strategies for multi-class SVM
- Effect of image flattening and pixel normalization on classifier input
- Confusion matrix, accuracy, precision, recall, and F1-score
- Python programming: NumPy, Pandas, Matplotlib, Scikit-learn basics

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

Complete the following tasks before arriving at the lab:

- Write the kernel function for the linear, polynomial, and RBF kernels.
- Explain what the degree parameter d controls in a polynomial kernel.
- Explain what the gamma parameter controls in an RBF kernel.
- Explain why SVM training time typically increases with dataset size and dimensionality, and why this matters for image data such as Fashion MNIST.
- State one advantage and one disadvantage of using a linear kernel versus a non-linear kernel.
- Explain how a multi-class classification problem (10 classes in Fashion MNIST) can be handled using a binary SVM formulation.
- Install required Python libraries: numpy, pandas, matplotlib, scikit-learn.

3. Theory

3.1 Recap: SVM and the Role of the Kernel

A Support Vector Machine classifies data by finding a hyperplane that maximizes the margin between classes. The decision function has the general form:

$$f(x) = \text{sign}\left(\Sigma\ \alpha_i\ y_i\ K(x_i, x) + b \right)$$

Here $K(x_i, x)$ is the kernel function, which implicitly computes a dot product in a (possibly much higher-

dimensional) transformed feature space without explicitly performing that transformation. The choice of kernel determines the shape of the decision boundary the SVM can represent.

3.2 Linear Kernel

$$K(x_i, x_j) = x_i^T x_j$$

The linear kernel produces a straight-line (or hyperplane) decision boundary in the original feature space. It is computationally efficient and works well when the classes are approximately linearly separable, but it cannot capture curved or complex decision boundaries.

3.3 Polynomial Kernel

$$K(x_i, x_j) = (\gamma \cdot x_i^T x_j + r)^d$$

The polynomial kernel introduces curved decision boundaries by implicitly considering polynomial combinations of the original features, controlled by the degree d. Higher degrees allow more complex boundaries but increase the risk of overfitting and can be computationally expensive to train.

3.4 RBF (Gaussian) Kernel

$$K(x_i, x_j) = \exp(-\gamma \, ||x_i - x_j||^2)$$

The Radial Basis Function (RBF) kernel measures similarity based on distance and can model highly non-linear, locally-varying decision boundaries. The gamma parameter controls how far the influence of a single training example reaches: a small gamma gives a smoother, more generalized boundary, while a large gamma produces a boundary that closely follows individual training points, increasing the risk of overfitting.

3.5 Comparing Kernel Trade-offs

Aspect	Linear	Polynomial	RBF
Decision boundary shape	Straight line / hyperplane	Curved, degree-dependent	Highly flexible, local
Training speed (large data)	Fastest	Slower, depends on degree	Slower than linear
Risk of overfitting	Low	Increases with degree	Increases with gamma
Interpretability	High (weights meaningful)	Moderate	Low (implicit feature space)
Best suited for	Linearly separable / high-dimensional sparse data	Data with known polynomial-like interactions	Complex, non-linear patterns (e.g. images)

3.6 Why Dimensionality Reduction Is Often Used on Image Data

Each Fashion MNIST image is 28×28 pixels, giving 784 raw features when flattened. SVM training time scales poorly with both the number of samples and the number of features, especially for non-linear kernels. Principal Component Analysis (PCA) is commonly applied beforehand to reduce dimensionality while retaining most of the variance, making SVM training on image data computationally feasible within a lab session.

3.7 Evaluation Metrics

Metric	Formula	Interpretation
Accuracy	(Correct predictions) / Total	Overall fraction of correctly classified images
Precision	TP / (TP + FP)	Of predicted class members, fraction actually correct
Recall	TP / (TP + FN)	Of actual class members, fraction correctly identified
F1-Score	2 · (Precision · Recall) / (Precision + Recall)	Harmonic mean of precision and recall

Training Time	Wall-clock seconds	Practical cost of fitting each kernel

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / Google Colab / VS Code
- Libraries: numpy, pandas, matplotlib, scikit-learn, tensorflow (or keras) for dataset loading
- Computer with minimum 8 GB RAM (recommended for image data and non-linear kernels)

5. Step-by-Step Procedure

Step 1: Import Libraries and Load Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time
from tensorflow.keras.datasets import fashion_mnist
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# Load Fashion MNIST dataset
(X_train_full, y_train_full), (X_test_full, y_test_full) = fashion_mnist.load_data()

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

print('Train shape:', X_train_full.shape)
print('Test shape:', X_test_full.shape)
```

Step 2: Exploratory Data Analysis

```
# Visualize a few sample images
plt.figure(figsize=(10, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_train_full[i], cmap='gray')
    plt.title(class_names[y_train_full[i]], fontsize=8)
    plt.axis('off')
plt.tight_layout()
plt.show()

# Class distribution
print(pd.Series(y_train_full).value_counts().sort_index())
```

Step 3: Preprocessing — Flatten, Subsample, Scale, and Reduce Dimensions

```
# SVM training on the full 60,000-image dataset with non-linear kernels is very slow,
# so a stratified subsample is used to keep the lab exercise tractable.
X_sub, _, y_sub, _ = train_test_split(
    X_train_full.reshape(len(X_train_full), -1), y_train_full,
    train_size=6000, stratify=y_train_full, random_state=42)

# Normalize pixel values to [0, 1]
X_sub = X_sub.astype('float32') / 255.0

# Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X_sub, y_sub, test_size=0.2, random_state=42, stratify=y_sub)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# Reduce 784 pixel features to 50 principal components to speed up SVM training
pca = PCA(n_components=50, random_state=42)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
print(f'Explained variance retained: {pca.explained_variance_ratio_.sum():.4f}')
```

Step 4: Train SVM with Linear Kernel

```
start = time.time()
svm_linear = SVC(kernel='linear', C=1.0)
svm_linear.fit(X_train_pca, y_train)
time_linear = time.time() - start

y_pred_linear = svm_linear.predict(X_test_pca)
```

Step 5: Train SVM with Polynomial Kernel

```
start = time.time()
svm_poly = SVC(kernel='poly', degree=3, C=1.0, gamma='scale')
svm_poly.fit(X_train_pca, y_train)
time_poly = time.time() - start

y_pred_poly = svm_poly.predict(X_test_pca)
```

Step 6: Train SVM with RBF Kernel

```
start = time.time()
svm_rbf = SVC(kernel='rbf', C=1.0, gamma='scale')
svm_rbf.fit(X_train_pca, y_train)
time_rbf = time.time() - start

y_pred_rbf = svm_rbf.predict(X_test_pca)
```

Step 7: Compare Kernel Performance

```
def evaluate(name, y_true, y_pred, train_time):
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, average='macro')
    rec = recall_score(y_true, y_pred, average='macro')
    f1 = f1_score(y_true, y_pred, average='macro')
    print(f'{name:12s} Acc={acc:.4f} Prec={prec:.4f} Rec={rec:.4f} F1={f1:.4f} Time={train_time:.2f}s')
    return [name, acc, prec, rec, f1, train_time]

results = []
results.append(evaluate('Linear', y_test, y_pred_linear, time_linear))
results.append(evaluate('Polynomial', y_test, y_pred_poly, time_poly))
results.append(evaluate('RBF', y_test, y_pred_rbf, time_rbf))

results_df = pd.DataFrame(results, columns=['Kernel', 'Accuracy', 'Precision', 'Recall', 'F1', 'Train
Time (s)'])
print(results_df)

results_df.plot(x='Kernel', y='Accuracy', kind='bar', legend=False, color='steelblue')
plt.ylabel('Test Accuracy')
plt.title('SVM Kernel Comparison on Fashion MNIST')
plt.tight_layout()
plt.show()
```

Step 8: Confusion Matrix for the Best-Performing Kernel

```
best_pred = y_pred_rbf  # replace with whichever kernel performed best
cm = confusion_matrix(y_test, best_pred)

plt.figure(figsize=(7, 6))
plt.imshow(cm, cmap='Blues')
plt.colorbar()
plt.xticks(range(10), class_names, rotation=90)
```

```
plt.yticks(range(10), class_names)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix – Best Kernel')
plt.tight_layout()
plt.show()
```

6. Expected Results & Observations

Kernel	Accuracy (approx.)	Relative Training Time	Notes
Linear	0.78 – 0.83	Fastest	Reasonable baseline; struggles with visually similar classes (e.g. Shirt vs Pullover vs Coat)
Polynomial (deg 3)	0.80 – 0.85	Moderate – Slow	Improves over linear on some classes; sensitive to degree and C
RBF	0.83 – 0.88	Moderate – Slow	Typically the best-performing kernel for image data; sensitive to gamma

Note: The RBF kernel is generally expected to outperform the linear kernel on Fashion MNIST, since the classes are not linearly separable in raw pixel space. Both non-linear kernels take noticeably longer to train than the linear kernel, and the polynomial kernel's performance is highly sensitive to the chosen degree. Most confusion is expected between visually similar upper-body garment classes (T-shirt/top, Pullover, Coat, Shirt).

7. Discussion Questions

- Which kernel achieved the highest test accuracy, and why might it be better suited to image data than the linear kernel?
- What are the main strengths and weaknesses of the linear kernel for this task?
- What are the main strengths and weaknesses of the polynomial kernel, and how does the degree parameter affect the trade-off between bias and variance?
- What are the main strengths and weaknesses of the RBF kernel, and how does gamma affect its tendency to overfit?
- Why was PCA applied before training the SVMs, and what would you expect to happen to training time and accuracy if it were skipped?
- Which Fashion MNIST classes were most frequently confused with each other, and why might that be the case?
- If you had to deploy one of these three models in a resource-constrained environment, which kernel would you choose and why?

8. Conclusion (Template for Student Record)

In this experiment, SVM classifiers with linear, polynomial, and RBF kernels were implemented and compared on the Fashion MNIST dataset:

- Number of PCA components used: _____ , retaining _____ % of variance.
- Polynomial kernel degree used: _____ .
- Best-performing kernel observed: _____ .

After preprocessing (subsampling, normalization, scaling, and PCA), all three models were trained and evaluated using accuracy, precision, recall, F1-score, and training time.

Kernel	Accuracy	Precision	Recall	F1-Score	Train Time (s)
Linear					
Polynomial					
RBF					

It was observed that _____ [student fills their observations, including which kernel performed best, the trade-off between accuracy and training time, and which classes were most frequently confused]. The experiment achieved the course outcomes CO1, CO2, CO3, and CO4 by implementing, analysing, and comparing multiple SVM kernels on real-world image data.

EXPERIMENT 13

Multilayer Feed-Forward Networks

Predicting Wine Quality using MLPs with Varying Architectures

Course	Machine Learning (ML Lab)
Experiment	13 — Multilayer Feed-Forward Network (MLP)
Dataset	Wine Quality Dataset (UCI / sklearn-compatible CSV)
Duration	3 Hours
Max. Marks	25 Marks

1. Aim

Implement and train a Multilayer Feed-Forward Neural Network (MLP) on the Wine Quality dataset to classify wines as good or not-good quality. The experiment covers loading and preprocessing the data, designing MLP architectures with varying numbers of hidden layers and neurons, training and evaluating each architecture, and discussing how these architectural choices affect network performance.

2. Prerequisites

2.1 Theoretical Background

Before attending the lab, students must have a clear understanding of the following:

- Basics of supervised learning and classification problems
- Structure of a perceptron and a Multilayer Feed-Forward Network (input, hidden, and output layers)
- Activation functions: sigmoid, tanh, ReLU, and softmax
- Forward propagation and how predictions are computed layer by layer
- Backpropagation and gradient descent as the basis of MLP training
- Effect of network depth (number of hidden layers) and width (neurons per layer) on model capacity
- Overfitting and underfitting in neural networks, and basic regularization strategies
- Python programming: NumPy, Pandas, Matplotlib, Scikit-learn basics

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

Complete the following tasks before arriving at the lab:

- Write the forward-pass equations for an MLP with one hidden layer, including the activation function.
- Explain why a non-linear activation function is necessary between layers of an MLP.
- Explain, at a conceptual level, how backpropagation updates the network's weights.
- Explain what is likely to happen if an MLP has too few neurons for a given task, and what is likely to happen if it has too many.
- Define overfitting in the context of neural networks and name one technique used to reduce it.
- State one reason feature scaling is important before training an MLP.
- Install required Python libraries: numpy, pandas, matplotlib, scikit-learn.

3. Theory

3.1 Perceptron and MLP Architecture

A Multilayer Feed-Forward Network (MLP) consists of an input layer, one or more hidden layers, and an output layer, where each neuron in a layer is connected to every neuron in the next layer. Unlike a single perceptron, which can only represent linearly separable functions, stacking multiple layers with non-linear activation functions allows an MLP to approximate complex, non-linear decision boundaries.

3.2 Forward Propagation

For a single hidden layer, the output is computed as:

$$\mathbf{h = g(W_1^T x + b_1), \quad \hat{y} = \sigma(W_2^T h + b_2)}$$

where W_1, b_1 are the weights and biases of the hidden layer, W_2, b_2 are the weights and biases of the output layer, $g(\cdot)$ is a hidden-layer activation function, and $\sigma(\cdot)$ is the output activation function (e.g., sigmoid for binary classification).

3.3 Activation Functions

Function	Formula	Typical Use
Sigmoid	$1 / (1 + e^{-x})$	Output layer for binary classification
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	Hidden layers (zero-centered output)
ReLU	$\max(0, x)$	Hidden layers (fast, avoids vanishing gradients)
Softmax	$e^{x_i} / \Sigma_j e^{x_j}$	Output layer for multi-class classification

3.4 Loss Function and Backpropagation

Training minimizes a loss function — typically binary cross-entropy for binary classification — by computing the gradient of the loss with respect to every weight using the chain rule (backpropagation), then updating weights via gradient descent:

$$\mathbf{W \leftarrow W - \eta \cdot \partial L / \partial W}$$

where η is the learning rate. This process is repeated over many epochs until the loss converges.

3.5 Effect of Hidden Layers and Neurons on Model Capacity

Architecture Choice	Effect on Capacity	Risk
Very few neurons / one small hidden layer	Low capacity; may fail to capture patterns in the data	Underfitting — high bias, poor training and test performance
Moderate hidden layers and neurons	Balanced capacity for many tabular datasets	Generally a good starting point; still requires tuning
Very many neurons / very deep networks	High capacity; can memorize training data	Overfitting — low training error but poor generalization; harder and slower to train

3.6 Regularization and Early Stopping

To control overfitting in larger networks, common strategies include L2 weight regularization, dropout (randomly deactivating neurons during training), and early stopping (halting training once validation performance stops improving). These techniques allow larger architectures to be used without a proportional increase in overfitting risk.

3.7 Evaluation Metrics

Metric	Formula	Interpretation
Accuracy	(TP + TN) / Total	Overall fraction of correct predictions
Precision	TP / (TP + FP)	Of predicted good-quality wines, fraction actually good
Recall	TP / (TP + FN)	Of actual good-quality wines, fraction correctly identified
F1-Score	2 · (Precision · Recall) / (Precision + Recall)	Harmonic mean of precision and recall

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / Google Colab / VS Code
- Libraries: numpy, pandas, matplotlib, scikit-learn

- Computer with minimum 4 GB RAM

5. Step-by-Step Procedure

Step 1: Import Libraries and Load Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# Load Wine Quality dataset (UCI repository – red wine variant)
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv'
df = pd.read_csv(url, sep=';')
print(df.head())
print(df['quality'].value_counts().sort_index())
```

Step 2: Exploratory Data Analysis

```
plt.figure(figsize=(7, 4))
plt.hist(df['quality'], bins=range(3, 10), color='steelblue', edgecolor='white', align='left')
plt.xlabel('Quality Score')
plt.ylabel('Count')
plt.title('Distribution of Wine Quality Scores')
plt.show()

print(df.describe())
print(df.isna().sum())
```

Step 3: Data Preprocessing

```
# Binarize target: quality >= 7 is 'good' (1), otherwise 'not good' (0)
df['good_quality'] = (df['quality'] >= 7).astype(int)

X = df.drop(columns=['quality', 'good_quality']).values
y = df['good_quality'].values

# Train-test split (80/20), stratified due to class imbalance
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Feature scaling (important for gradient-based MLP training)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Step 4: Train a Baseline MLP

```
baseline_mlp = MLPClassifier(hidden_layer_sizes=(8,), activation='relu',
                             max_iter=500, random_state=42)
baseline_mlp.fit(X_train_scaled, y_train)

y_pred_base = baseline_mlp.predict(X_test_scaled)
print(f'Baseline (8,) => Accuracy={accuracy_score(y_test, y_pred_base):.4f}')
```

Step 5: Experiment with Varying Architectures

```
architectures = [
    (4,),            # single small hidden layer
    (16,),           # single moderate hidden layer
    (64,),           # single large hidden layer
    (16, 8),         # two hidden layers
    (32, 16, 8),     # three hidden layers
]

results = []
models = {}
```

```
for arch in architectures:
    mlp = MLPClassifier(hidden_layer_sizes=arch, activation='relu',
                        max_iter=1000, random_state=42)
    mlp.fit(X_train_scaled, y_train)
    y_pred = mlp.predict(X_test_scaled)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)

    results.append((str(arch), acc, prec, rec, f1, mlp.n_layers_))
    models[str(arch)] = mlp
    print(f'{str(arch):15s} Acc={acc:.4f} Prec={prec:.4f} Rec={rec:.4f} F1={f1:.4f}')
```

Step 6: Compare Architectures

```
results_df = pd.DataFrame(results, columns=['Architecture', 'Accuracy', 'Precision', 'Recall', 'F1',
'Layers'])
print(results_df)

plt.figure(figsize=(8, 5))
plt.bar(results_df['Architecture'], results_df['Accuracy'], color='steelblue')
plt.ylabel('Test Accuracy')
plt.xlabel('Hidden Layer Architecture')
plt.title('Effect of MLP Architecture on Test Accuracy')
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

Step 7: Visualize Training Loss Curve for the Best Model

```
best_arch = results_df.loc[results_df['F1'].idxmax(), 'Architecture']
best_model = models[best_arch]

plt.figure(figsize=(7, 4))
plt.plot(best_model.loss_curve_, color='crimson')
plt.xlabel('Epoch')
plt.ylabel('Training Loss')
plt.title(f'Training Loss Curve — Best Architecture {best_arch}')
plt.grid(alpha=0.3)
plt.show()
```

Step 8: Confusion Matrix for the Best Architecture

```
y_pred_best = best_model.predict(X_test_scaled)
cm = confusion_matrix(y_test, y_pred_best)

print(f'Best architecture: {best_arch}')
print('Confusion Matrix:')
print(cm)
```

6. Expected Results & Observations

Architecture	Expected Accuracy (approx.)	Notes
(4,) — very small	0.85 – 0.88	Limited capacity; may underfit, especially on precision/recall for the minority 'good' class
(16,) — moderate	0.87 – 0.90	Often a good balance for this dataset size and feature count
(64,) — large single layer	0.87 – 0.90	Similar to moderate; may start overfitting slightly with more training epochs
(16, 8) — two layers	0.87 – 0.91	Additional depth can help capture feature interactions

(32, 16, 8) — three layers	0.86 – 0.90	Higher capacity; risk of overfitting increases without regularization or early stopping

Note: The Wine Quality dataset has a strong class imbalance (few wines rated 'good'), so overall accuracy alone can be misleading — precision and recall on the minority class are important to examine. Very small architectures tend to underfit and predict the majority class too often, while very large architectures may fit the training data closely without a proportional gain on the test set.

7. Discussion Questions

- Which architecture achieved the best F1-score, and how does it compare to the baseline in terms of accuracy and precision/recall on the minority class?
- What evidence in your results suggests underfitting for the smallest architecture?
- What evidence in your results suggests overfitting for the largest architecture, if any?
- How did adding a second or third hidden layer affect performance compared to simply widening a single hidden layer?
- Why is class imbalance in the Wine Quality dataset important to consider when interpreting accuracy?
- What regularization or early-stopping strategy would you try next to improve the larger architectures?
- If deploying this model, would you prioritize the highest accuracy or the highest recall on 'good' quality wines? Why?

8. Conclusion (Template for Student Record)

In this experiment, Multilayer Feed-Forward Networks with varying architectures were implemented and compared on the Wine Quality dataset:

- Architectures tested: _____ , _____ , _____ , _____ , and _____ .
- Best-performing architecture observed: _____ .
- Activation function used: _____ .

After preprocessing (binarizing the quality label, scaling, and train-test split), each architecture was trained and evaluated using accuracy, precision, recall, and F1-score.

Architecture	Accuracy	Precision	Recall	F1-Score
(4,)				
(16,)				
(64,)				
(16, 8)				
(32, 16, 8)				

It was observed that _____ [student fills their observations, including which architecture generalized best, signs of underfitting or overfitting, and how depth versus width affected performance]. The experiment achieved the course outcomes CO1, CO2, CO3, and CO4 by implementing, analysing, and comparing MLP architectures on real-world data.

EXPERIMENT 14
Neural Network Performance Analysis Using Variant Activation Functions

Convergence Profiles and Classification Accuracy Evaluation on the MNIST Dataset

Course	Machine Learning (ML Lab)
Experiment	14 — Performance Analysis of Neural Network Activation Functions
Dataset	MNIST Handwritten Digit Dataset (OpenML / fetch_openml)
Duration	3 Hours

1. Aim

Implement a neural network classifier using different activation functions (Sigmoid, ReLu, Tanh) on the MNIST dataset. Analyze how each activation function affects the training process and classification accuracy.

2. Prerequisites

2.1 Theoretical Background

- Supervised artificial neural network architectures (Layers, Nodes, Weights, Biases).
- Non-linear mapping metrics: Activation functions as structural non-linear continuous processing transforms.
- Backpropagation mechanics: Gradient computation via chain-rule optimization routines.
- The vanishing gradient problem and its relationship to activation saturation outside center thresholds.

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

- Write the complete mathematical expression for the Sigmoid, Tanh, and ReLU activation functions.
- Define the formula for the analytical derivative of the Sigmoid activation function and specify its maximum value.
- Explain why standard deep neural networks suffer from the vanishing gradient problem when using saturating activations.
- Define what a zero-centered activation framework signifies within hidden layer preprocessing, detailing the behavioral benefit of Tanh over Sigmoid.
- Why does the basic ReLU algorithm demonstrate structural optimization speed improvements, and how does it avoid the vanishing gradient trap?

3. Theory

3.1 Neural Network Architecture and Activation Mechanics

A Feedforward Deep Neural Network (Multi-Layer Perceptron) processes complex image tensors by passing them through successive computational layers. For a multi-class image recognition task like the MNIST handwritten digits dataset, each neuron computes a weighted combination of its inputs plus a bias parameter. However, without non-linearity, a multi-layer network collapses into a single linear transformation, rendering it incapable of tracking highly non-linear decision spaces. Activation functions introduce this non-linearity, allowing the model to map intricate geometric boundaries.

3.2 The Mathematics of Core Activation Functions

During forward propagation, hidden layer inputs are transformed into output signals using specific activation functions, each possessing unique derivative mathematical profiles:

1. Sigmoid (Logistic) Function:

The Sigmoid function maps real-valued numbers into a strictly bounded probability range between 0 and 1. It is useful for interpreting probabilistic distributions but suffers from severe gradient compression outside its center threshold.

Mathematical Formula: f(x) = 1 / (1 + e^-x)

Analytical Derivative: f'(x) = f(x) * (1 - f(x))

2. Hyperbolic Tangent (Tanh) Function:

The Tanh function maps inputs into a range between -1 and +1, making it zero-centered. This means its steady-state empirical mean tracks closely to zero, preventing systematic weight-update biases during backpropagation. However, it still saturates at extreme input ranges.

Mathematical Formula: f(x) = (e^x - e^-x) / (e^x + e^(-x))

Analytical Derivative: f'(x) = 1 - f(x)^2

3. Rectified Linear Unit (ReLU) Function:

The ReLU activation function is a piecewise linear transformation that passes positive values through unchanged while completely blocking negative values. Because its derivative is a constant 1.0 for all positive inputs, gradients propagate freely through deep structures without being shrunk by fractional scaling multipliers.

Mathematical Formula: f(x) = max(0, x)

Analytical Derivative: f'(x) = 1 if x > 0 else 0

3.3 The Vanishing Gradient Problem and Convergence Speed

During backpropagation, error gradients are multiplied down from the final classification layer back to initial weight layers via the chain rule. For the Sigmoid function, the maximum possible value of its derivative is 0.25. As these fractions are repeatedly multiplied together across multiple layers, the gradient vanishes exponentially:

Gradient Scaling Factor ≈ $\prod$ (w_i * f'(x_i))

When this value approaches zero, parameter updates stall entirely, preventing the network from minimizing its cross-entropy loss. In contrast, the constant derivative of the ReLU function maintains steady gradient flow, enabling fast training convergence and scaling smoothly to deeper architectures.

- • **Analyzing Model Behavior:** Tracking optimization profiles yields white-box convergence data. Monitoring validation metrics over sequential iterations isolates training latency profiles and highlights the exact epoch where gradient propagation stalls.
- • **Architectural Comparison:** Layers utilizing activation models that avoid optimization saturation yield sustained gradient flows, driving structural reduction in cross-entropy classification impurity while reducing computational overhead.

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / VS Code Workspace
- Libraries: numpy, pandas, scikit-learn, matplotlib
- Hardware: Minimum 8GB RAM

5. Step-by-Step Procedure

Step 1: Load and Preprocess the MNIST Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load MNIST data
mnist = fetch_openml('mnist_784', version=1, as_frame=False, parser='auto')
X, y = mnist.data / 255.0, mnist.target.astype(int)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.15, random_state=42)
print("Dataset processing complete.")
```

Step 2: Train Neural Network Architecture Using Variant Activation Metrics

```
history_loss = {}
```

```
test_accuracies = {}

activations = ['logistic', 'tanh', 'relu']
for act in activations:
    print(f"Training network using activation: {act}")
    mlp = MLPClassifier(hidden_layer_sizes=(128, 64), activation=act, max_iter=25,
random_state=42)
    mlp.fit(X_train, y_train)
    history_loss[act] = mlp.loss_curve_
    test_accuracies[act] = accuracy_score(y_test, mlp.predict(X_test))
```

Step 3: Structural Plot Generation and Convergence Profiling

```
plt.figure(figsize=(10, 5))
for act in activations:
    plt.plot(history_loss[act], label=act)
plt.title('Loss Convergence Comparison')
plt.xlabel('Epochs')
plt.ylabel('Loss Value')
plt.legend()
plt.show()
```

6. Expected Results & Observations

The following table:

Extracted Feature	Relative Convergence Speed	Observed Test Accuracy	Empirical Behavioral Signal Analysis
Sigmoid Function	Slowest Path	88% - 92%	Demonstrates structural gradient saturation at processing tails.
Tanh Function	Moderate Path	93% - 95%	Zero-centered layout increases early optimization step efficiency.
ReLU Function	Fastest Path	96% - 98%+	Constant derivative prevents gradient degradation across active nodes.

7. Discussion Questions

- Explain how a visual path of a loss convergence plot directly maps to an actionable, rule-based training modification strategy.
- Why does the Sigmoid criteria struggle when handling deep multi-layered hidden architectures, and how does ReLU mitigate this limitation?
- If a neural network runs with an completely unrestricted iteration parameter, what happens to the training vs. testing accuracy profile?
- Explain the concept of network overfitting.
- What specific architectural indicators from the convergence plot would guide an optimization manager to implement early stopping criteria?

8. Conclusion (Template for Student Record)

In this laboratory experiment, a multi-layer Neural Network framework was constructed over handwritten digit profiles. The model parsed pixel layers to discover optimization dynamics and performance profiles.

The following table:

Behavioral Feature Column Name	**Observed Convergence Speed Score**	**Observed Test Target Accuracy %**
Sigmoid Activation Layer		
Tanh Activation Layer		
ReLU Activation Layer		

EXPERIMENT 15
Hyperparameter Optimization and Empirical Tuning Profiles in Deep Neural Networks

Grid Search Grid-Analysis on Learning Rates, Batch Sizes, and Epoch Horizons using the Fashion MNIST Dataset

Course	Machine Learning (ML Lab)
Experiment	15 — Hyperparameter Tuning and Grid Search Optimization in Neural Networks
Dataset	Fashion MNIST Dataset (Zalando Research Article Images)
Duration	3 Hours

1. Aim

Implement a Deep Neural Network (Multi-Layer Perceptron) classifier on the Fashion MNIST dataset, execute a multi-dimensional hyperparameter grid sweep across variant learning rates, batch sizes, and training epochs, and evaluate how these continuous and discrete structural knobs dictate cross-entropy loss convergence, execution latency, and test generalization.

2. Prerequisites

2.1 Theoretical Background

- Hyperparameter Spaces vs. Model Parameters: Parameters (weights and biases) are optimized internally via backpropagation; hyperparameters (learning rate, batch size, epoch budgets) must be external structural definitions.
- Stochastic Gradient Descent (SGD) and Learning Rate: The learning rate acts as a step-size multiplier scaling gradient steps. Suboptimal choices trigger either oscillatory divergence or prolonged stagnation.
- Mini-batch Optimization Dynamics: Mini-batch sizes establish the tension between the pure gradient tracking of full-batch training and the high-variance regularizing noise of single-instance updates.
- Epoch Budgets and Overfitting Profiles: An epoch represents a complete forward-backward pass through the training data. Excessive epoch budgets without regularizers introduce variance-driven generalization decline.

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

- Differentiate clearly between model parameters and structural architectural hyperparameters in an ANN.
- Explain the mathematical and behavioral consequences on a loss surface if the learning rate is initialized (a) excessively large, or (b) excessively small.
- Why does increasing the training batch size typically reduce the training runtime per epoch while potentially degrading final generalization performance?
- Define the term Overfitting and describe how tracking training vs. validation loss across sequential epochs reveals its onset.
- Detail how the Fashion MNIST dataset structurally differs from the standard digits MNIST dataset in classification complexity.

3. Theory

3.1 Neural Network Hyperparameter Controls and Vector Mechanics

A multi-layered feedforward neural network extracts non-linear representations by passing tensor matrices through optimized weight vectors. The efficiency of this mapping relies entirely on regularized parameter searching. The learning rate acts as a foundational optimization scalar within the gradient descent step, dictating the step size towards the global minimum. Mini-batch sizing establishes the empirical gradient estimation vector length, balancing speed against gradient variance noise, while the epoch count sets the overall optimization horizon budget.

3.2 Hyperparameter Tuning Metrics and Convergence Profiles

Altering optimization configurations changes the behavioral trajectory of the cross-entropy objective function.

Systematic tracking yields deep insight into model efficiency.

The following table:

Target Knobs	Low Value Signature	High Value Signature	Empirical Balance Objective
Learning Rate	Stagnant, linear loss trajectories; slow training.	High numerical oscillations; infinite loss divergence.	Traverses complex saddle spaces without bouncing over minimums.
Batch Size	Noisy updates; high hardware latency.	Smooth updates; large memory usage; generalization drops.	Exploits hardware vector parallelization without trapping in local traps.
Epoch Budget	Underfitted boundary tracking.	Overfitted boundary memorization.	Halts precisely at the convergence validation trough.

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / VS Code Workspace
- Libraries: numpy, pandas, scikit-learn, matplotlib
- Hardware: Minimum 8GB RAM

5. Step-by-Step Procedure

Step 1: Load and Preprocess the Fashion MNIST Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Fetch Fashion MNIST dataset
fashion_mnist = fetch_openml('Fashion-MNIST', version=1, as_frame=False, parser='auto')
X, y = fashion_mnist.data / 255.0, fashion_mnist.target.astype(int)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, stratify=y,
random_state=42)
# Subsample for laboratory execution limits
X_train_sub, _, y_train_sub, _ = train_test_split(X_train, y_train, train_size=10000,
stratify=y_train, random_state=42)
```

Step 2: Implement Hyperparameter Grid-Sweep Analysis

```
learning_rates = [0.001, 0.1]
batch_sizes = [32, 256]
epoch_limits = [10, 20]
results_registry = []

for lr in learning_rates:
    for bs in batch_sizes:
        for epochs in epoch_limits:
            mlp = MLPClassifier(hidden_layer_sizes=(128, 64), activation='relu', solver='sgd',
                                learning_rate_init=lr, batch_size=bs, max_iter=epochs,
random_state=42)
            start_time = time.time()
            mlp.fit(X_train_sub, y_train_sub)
            elapsed = time.time() - start_time
            test_acc = accuracy_score(y_test, mlp.predict(X_test))
```

```
            results_registry.append({'LR': lr, 'Batch': bs, 'Epochs': epochs, 'Time': elapsed,
'Accuracy': test_acc * 100})
```

Step 3: Performance Analysis and Visualization

```
summary_df = pd.DataFrame(results_registry)
print(summary_df.to_string(index=False))

plt.figure(figsize=(10, 5))
configs = [f"LR:{r['LR']}|B:{r['Batch']}|E:{r['Epochs']}" for r in results_registry]
accuracies = [r['Accuracy'] for r in results_registry]
plt.barh(configs, accuracies, color='steelblue')
plt.xlabel('Test Accuracy (%)')
plt.title('Hyperparameter Configuration vs Accuracy')
plt.show()
```

6. Expected Results & Observations

The following table:

Extracted Feature	Relative Convergence Speed	Observed Test Accuracy	Empirical Behavioral Signal Analysis
High LR (0.1), Small Batch (32)	Rapid initial descent but highly erratic.	78% - 82%	Excessive step size paired with noisy mini-batch samples causes the model to overshoot optimal minimums.
Low LR (0.001), Large Batch (256)	Extremely slow, near-linear decline.	60% - 70%	Small step sizes combined with averaged gradients lead to optimization stagnation over short epoch budgets.
Balanced Configuration	Smooth, logarithmic convergence path.	84% - 88%+	Properly scaled learning rates combined with stable mini-batch sizes allow regularized generalization to emerge.

7. Discussion Questions

- Explain how a visual path of a loss convergence plot directly maps to an actionable, rule-based training modification strategy.
- Why does utilizing an excessively large batch size (e.g., 2048) often lead to a reduction in generalization accuracy compared to smaller batch sizes?
- If a neural network runs with an completely unrestricted iteration parameter, what happens to the training vs. testing accuracy profile? Explain the concept of network overfitting.
- What specific architectural indicators from a convergence plot guide an optimization manager to implement early stopping criteria?
- How does the choice of learning rate schedule (e.g., constant vs. adaptive decay) stabilize optimization paths over complex, non-convex loss surfaces?

8. Conclusion (Template for Student Record)

In this laboratory experiment, a systematic multi-dimensional hyperparameter grid sweep was successfully executed across variant learning rates, batch sizes, and epoch limits on a feedforward neural network classifying Fashion MNIST samples.

The following table:

Behavioral Feature Profile	**Target Learning Rate**	**Target Batch Size**	**Total Epochs**	**Observed Time (Sec)**	**Observed Test Accuracy %**
Underfitting Profile					
Overfitting / Erratic Profile					
Optimal Config Profile					

EXPERIMENT 16

Comparative Clustering Analysis: Partitional (K-Means) vs. Hierarchical (Agglomerative)

Customer Segmentation and Unsupervised Validation Frameworks on the Mall Customer Dataset

Course	Machine Learning (ML Lab)
Experiment	16 — Partitional & Hierarchical Clustering Analysis
Dataset	Mall Customer Dataset (UCI Repository / Sklearn Synthetic Custom Layout)
Duration	3 Hours
Max. Marks	25 Marks

1. Aim

Implement and compare partitional (K-Means) and hierarchical (Agglomerative) clustering algorithms on the Mall Customer dataset to group consumers into spending profiles. The experiment covers hyperparameter exploration via the Elbow Method (Inertia criteria) and Dendrogram visualization, followed by an architectural performance comparison using Silhouette Coefficients and cluster geometry plots.

2. Prerequisites

2.1 Theoretical Background

- Unsupervised learning paradigms and the lack of predefined ground-truth training vectors.
- Geometric distance modeling and the mathematical formulation of centroid-based vs. linkage-based partitions.
- The core concept of cluster density, compactness, and separation boundaries.
- Internal validation metrics: Within-Cluster Sum of Squares (Inertia) and the Silhouette Coefficient range [-1, +1].
- Python programming structures using NumPy, Pandas, Matplotlib, and Scikit-Learn unsupervised modules.

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

- Write the mathematical definition for the objective cost function (Inertia) minimized by K-Means clustering.
- Write the complete formula for the Silhouette Coefficient of an individual data point, defining both the cohesion and separation parameters.
- Explain the key operational difference between Ward's linkage, single linkage, and complete linkage criteria in hierarchical clustering.
- What does the geometric 'Elbow' indicate in a Within-Cluster Sum of Squares (WCSS) plot vs. Number of Clusters graph?
- Why is multi-dimensional feature scaling (e.g., StandardScaler) strictly mandatory before applying distance-based clustering models?
- Explain why K-Means might converge to a sub-optimal local minimum, and how the 'k-means++' initialization strategy mitigates this vulnerability.

3. Theory

3.1 Partitional Clustering (K-Means Paradigm)

K-Means is an iterative centroid-based partition technique that decomposes an unlabelled dataset into K distinct, non-overlapping groups. Given a choice for K, the algorithm alternates between assigning points to their nearest centroid based on Euclidean distance, and updating centroids to be the mean of their assigned points. The model minimizes the Within-Cluster Sum of Squares (WCSS):

WCSS (Inertia) = sum_{k=1}^{K} sum_{x in C_k} ||x - mu_k||^2

3.2 Hierarchical Clustering (Agglomerative Approach)

Agglomerative clustering uses a bottom-up connectivity approach where each observation starts in its own unique cluster. In each successive step, the two closest clusters are merged until all elements form a single tree structure. Ward's linkage is often chosen to minimize the total within-cluster variance increase during a merge action:

Delta ESS_{A, B} = (|A|*|B| / (|A| + |B|)) * ||mu_A - mu_B||^2

3.3 Evaluation Metrics Profile

The following table:

Metric	Mathematical Formulation	Interpretation Criteria
Inertia (WCSS)	sum_{k} sum_{x in C_k} ||x - mu_k||^2	Lower values indicate tighter, more compact clusters; used to build elbow paths.
Silhouette Score	(b_i - a_i) / max(a_i, b_i)	Ranges from -1 to +1. Values near +1 indicate excellent cohesion and clean cluster separation.
Dendrogram Height	Linkage distance metric threshold	Represents the structural dissimilarity at which sub-clusters are merged.

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / Google Colab / VS Code Workspace
- Libraries: numpy, pandas, matplotlib, scikit-learn, scipy
- Computer workstation with minimum 4 GB RAM

5. Step-by-Step Procedure

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
import scipy.cluster.hierarchy as sch

# Step 1: Import Libraries and Load Dataset Simulation
np.random.seed(42)
n_samples = 200
mall_data = {
    'CustomerID': range(1, n_samples + 1),
    'Age': np.random.randint(18, 70, n_samples),
    'Annual_Income_k': np.random.randint(15, 135, n_samples),
    'Spending_Score': np.random.randint(1, 100, n_samples)
}
df = pd.DataFrame(mall_data)
print("Dataset Overview:\n", df.head())

# Step 2 & 3: Isolate Features and Apply Preprocessing (Scaling)
X = df[['Annual_Income_k', 'Spending_Score']].values
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 4: Partitional Clustering Optimization (K-Means & Elbow Method)
wcss = []
k_values = range(2, 11)
for k in k_values:
    km = KMeans(n_clusters=k, init='k-means++', random_state=42, n_init=10)
    km.fit(X_scaled)
```

```
    wcss.append(km.inertia_)

# Fit final optimal K-Means model (K=5 based on standard elbow inflection)
optimal_km = KMeans(n_clusters=5, init='k-means++', random_state=42, n_init=10)
kmeans_labels = optimal_km.fit_predict(X_scaled)
kmeans_sil = silhouette_score(X_scaled, kmeans_labels)

# Step 5: Hierarchical Agglomerative Clustering
optimal_agg = AgglomerativeClustering(n_clusters=5, metric='euclidean', linkage='ward')
agg_labels = optimal_agg.fit_predict(X_scaled)
agg_sil = silhouette_score(X_scaled, agg_labels)

# Step 6 & 7: Print Metrics & Compare Profiles
print(f'\nK-Means (K=5) WCSS (Inertia): {optimal_km.inertia_:.2f}')
print(f'K-Means (K=5) Silhouette Score: {kmeans_sil:.4f}')
print(f'Agglomerative (K=5) Silhouette Score: {agg_sil:.4f}')
```

6. Expected Results & Observations

The following table:

Model Framework Structure	Inertia Metric (WCSS)	Observed Silhouette Metric	Geometric Cluster Density & Alignment
Linear Partitional K-Means	65.0 – 75.0 (Scaled)	0.55 – 0.58	Generates spherical, cleanly partitioned customer spending clusters.
Agglomerative Hierarchical	N/A (No Centroid Anchor)	0.53 – 0.56	Identifies highly similar boundaries with slight variations at density margins.

7. Discussion Questions

- Why is feature scaling highly important before applying distance-based clustering layouts like K-Means or Agglomerative models?
- Which behavioral clusters did K-Means and Agglomerative formats define? What does this imply about consumer spending tiers?
- How does the target value of K selected via the Elbow approach alter the model's structural bias-variance trade-off profile?
- Under what specific geometric conditions (e.g., nested circles, non-convex shapes) would you expect Agglomerative loops to outperform K-Means, and vice versa?
- Why might partitional and hierarchical techniques yield highly similar silhouette metrics on this specific consumer dataset?
- How would a hybrid framework combining a rapid K-Means pass with a downstream hierarchical tree assembly scale across massive multi-million row datasets?

8. Conclusion (Template for Student Record)

In this experiment, K-Means and Agglomerative Hierarchical clustering frameworks were implemented and evaluated against an unsupervised baseline layout:

1. K-Means clustering established a centroid-driven partitional framework with optimal K = _____ .

2. Agglomerative Hierarchical clustering constructed an iterative bottom-up linkage tree with target clusters = _____ .

The dataset utilized was the Mall Customer Dataset with target features of Annual Income and Spending Score.

The following table:

Model Framework	**Optimized Cluster Count**	**Final Scaled Inertia**	**Final Silhouette Score**
K-Means Partitional			
Agglomerative Hierarchical		N/A	

EXPERIMENT 17
K-Means Clustering

Applying and Evaluating K-Means Clustering on the Digits Dataset

Course	Machine Learning (ML Lab)
Experiment	17 — K-Means Clustering
Dataset	Digits Dataset (sklearn.datasets.load_digits — 8×8 handwritten digit images)
Duration	3 Hours
Max. Marks	25 Marks

1. Aim

Implement and apply K-Means clustering to the Digits dataset. The experiment covers loading and preprocessing the data, applying K-Means with varying numbers of clusters, evaluating clustering performance using inertia and silhouette score, and analysing how the choice of the number of clusters (K) affects clustering quality.

2. Prerequisites

2.1 Theoretical Background

Before attending the lab, students must have a clear understanding of the following:

- Basics of unsupervised learning and the goal of clustering
- Distance and similarity measures, particularly Euclidean distance
- The K-Means algorithm: centroid initialization, assignment step, and update step
- The concept of inertia (within-cluster sum of squared distances)
- The concept of silhouette score and cluster cohesion versus separation
- The elbow method for selecting an appropriate value of K
- Basics of dimensionality reduction (PCA) for visualizing high-dimensional clusters
- Python programming: NumPy, Pandas, Matplotlib, Scikit-learn basics

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

Complete the following tasks before arriving at the lab:

- Explain the K-Means algorithm in your own words, including the assignment and update steps.
- Explain what inertia measures, and why it always decreases as the number of clusters increases.
- Explain what the silhouette score measures and state its range of possible values.
- Explain the elbow method and how it is used to select an appropriate value of K.
- Explain whether feature scaling is necessary before applying K-Means to pixel-intensity data, and why.
- Describe one limitation of K-Means (e.g., sensitivity to initialization, assumption of spherical clusters).
- Install required Python libraries: numpy, pandas, matplotlib, scikit-learn.

3. Theory

3.1 Overview of Clustering and K-Means

Clustering is an unsupervised learning task that groups data points such that points within a group (cluster) are more similar to each other than to points in other groups. K-Means is a centroid-based clustering algorithm that partitions a dataset into K clusters by minimizing the total within-cluster variance, without requiring any labelled data.

3.2 K-Means Algorithm Steps

The K-Means algorithm proceeds iteratively as follows:

- Initialize K cluster centroids, typically using the k-means++ strategy for better starting positions.
- Assignment step: assign each data point to the nearest centroid based on Euclidean distance.
- Update step: recompute each centroid as the mean of all points assigned to it.
- Repeat the assignment and update steps until centroids stabilize or a maximum number of iterations is reached.

The algorithm minimizes the following objective function, where C_i is the set of points in cluster i and μ_i is its centroid:

$$J = \Sigma_i \Sigma(x \in C_i) \, ||x - \mu_i||^2$$

3.3 Choosing the Number of Clusters (K)

The Digits dataset contains images of handwritten digits from 0 to 9, so a natural choice is K = 10. However, K-Means does not use the true labels during training, so it is instructive to experiment with several values of K and evaluate the resulting clusters quantitatively rather than assuming K = 10 will always give the best-separated clusters.

3.4 Evaluation Metric: Inertia

Inertia is the sum of squared distances between each sample and its assigned cluster centroid. Lower inertia indicates tighter, more compact clusters. However, inertia decreases monotonically as K increases (approaching zero when K equals the number of samples), so it cannot be used alone to select K — it is typically combined with the elbow method, which looks for the point where the rate of decrease slows sharply.

3.5 Evaluation Metric: Silhouette Score

The silhouette score measures how similar a point is to its own cluster compared to other clusters. For a sample, let a be the mean distance to other points in the same cluster, and b be the mean distance to points in the nearest neighbouring cluster. The silhouette coefficient for that sample is:

$$s = (b - a) / \max(a, b)$$

The silhouette score ranges from −1 to +1. Values close to +1 indicate well-separated, dense clusters; values near 0 indicate overlapping clusters; and negative values suggest samples may have been assigned to the wrong cluster. The overall silhouette score is the mean of the coefficient across all samples.

3.6 Elbow Method and Silhouette Analysis

In practice, the number of clusters is chosen by plotting inertia against K (the elbow curve) and separately plotting the silhouette score against K. The elbow point — where inertia stops decreasing rapidly — combined with the K value that maximizes the silhouette score, gives a well-supported estimate of a suitable number of clusters.

3.7 Limitations of K-Means

- Requires the number of clusters K to be specified in advance.
- Sensitive to the initial placement of centroids; poor initialization can lead to poor local optima.
- Assumes roughly spherical, equally sized clusters, which may not match the true data structure.
- Sensitive to feature scale and to outliers.
- Can converge to a local optimum rather than the global optimum of the objective function.

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / Google Colab / VS Code
- Libraries: numpy, pandas, matplotlib, scikit-learn
- Computer with minimum 4 GB RAM

5. Step-by-Step Procedure

Step 1: Import Libraries and Load Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Load the Digits dataset (8x8 grayscale images of handwritten digits 0-9)
digits = load_digits()
X = digits.data
y = digits.target
print(X.shape, y.shape)
print(np.unique(y, return_counts=True))
```

Step 2: Exploratory Data Analysis

```
fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for i, ax in enumerate(axes.flat):
    ax.imshow(digits.images[i], cmap='gray')
    ax.set_title(f'Label: {digits.target[i]}')
    ax.axis('off')
plt.tight_layout()
plt.show()

print(pd.Series(y).value_counts().sort_index())
```

Step 3: Data Preprocessing

```
# Scale features (each pixel intensity is 0-16; standardize before clustering)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Reduce to 2 components with PCA, used only for visualization
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)
print(f'Explained variance (2 components): {pca.explained_variance_ratio_.sum():.4f}')
```

Step 4: Train a Baseline K-Means Model (K = 10)

```
baseline_kmeans = KMeans(n_clusters=10, init='k-means++', n_init=10, random_state=42)
baseline_labels = baseline_kmeans.fit_predict(X_scaled)

baseline_inertia = baseline_kmeans.inertia_
baseline_sil = silhouette_score(X_scaled, baseline_labels)
print(f'Baseline (K=10) => Inertia={baseline_inertia:.2f}, Silhouette={baseline_sil:.4f}')
```

Step 5: Experiment with Varying Number of Clusters

```
k_values = [2, 4, 6, 8, 10, 12, 15, 20]
inertias = []
sil_scores = []

for k in k_values:
    kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
```

```
    labels = kmeans.fit_predict(X_scaled)
    inertias.append(kmeans.inertia_)
    sil = silhouette_score(X_scaled, labels)
    sil_scores.append(sil)
    print(f'K={k:2d}  Inertia={kmeans.inertia_:10.2f}  Silhouette={sil:.4f}')
```

Step 6: Plot the Elbow Curve and Silhouette Scores

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(k_values, inertias, marker='o', color='steelblue')
axes[0].set_xlabel('Number of Clusters (K)')
axes[0].set_ylabel('Inertia')
axes[0].set_title('Elbow Method: Inertia vs K')

axes[1].plot(k_values, sil_scores, marker='o', color='crimson')
axes[1].set_xlabel('Number of Clusters (K)')
axes[1].set_ylabel('Silhouette Score')
axes[1].set_title('Silhouette Score vs K')

plt.tight_layout()
plt.show()
```

Step 7: Visualize Cluster Centers as Images (K = 10)

```
fig, axes = plt.subplots(2, 5, figsize=(10, 4))
centers = baseline_kmeans.cluster_centers_.reshape(10, 8, 8)
for i, ax in enumerate(axes.flat):
    ax.imshow(centers[i], cmap='gray')
    ax.set_title(f'Cluster {i}')
    ax.axis('off')
plt.tight_layout()
plt.show()
```

Step 8: Visualize Clusters in 2D Using PCA

```
plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=baseline_labels, cmap='tab10', s=15)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('K-Means Clusters (K=10) Visualized via PCA')
plt.colorbar(scatter, label='Cluster')
plt.show()
```

6. Expected Results & Observations

K (Clusters)	Expected Inertia (approx.)	Expected Silhouette	Notes
2	Very high	0.15 – 0.20	Too few clusters; visually distinct digits are merged together.
4	High	0.15 – 0.20	Some digit groups combined; still too coarse to separate 10 classes.
6	Moderate–high	0.15 – 0.20	Improves grouping but several digits still share a cluster.
8	Moderate	0.16 – 0.20	Closer to natural structure; a couple of digits may remain merged.
10	Moderate (elbow region)	0.18 – 0.22	Matches the true number of digit classes; typically the best balance of inertia and silhouette.

12	Slightly lower	0.16 – 0.20	Some clusters begin to split a single digit into two sub-clusters.
15	Lower	0.14 – 0.18	Over-segmentation; silhouette score starts declining as clusters get smaller and less separated.
20	Lowest of the set	0.12 – 0.16	Clear over-clustering; many small, less meaningful clusters.

Note: Inertia decreases continuously as K increases and cannot indicate the best K by itself. The silhouette score typically peaks near K = 10, matching the true number of digit classes, then declines as clusters are split unnecessarily. Very small values of K under-segment visually distinct digits, while very large values of K over-segment the data into clusters that no longer correspond to whole digit classes.

7. Discussion Questions

- Which value of K produced the highest silhouette score, and how does it compare to K = 10, the true number of digit classes?
- Why does inertia always decrease as K increases, even when the clustering quality does not actually improve?
- What does a silhouette score close to zero indicate about the overlap between clusters?
- How did visualizing the cluster centers as images help you judge whether the clusters correspond to meaningful digit groups?
- What evidence in your results suggests that some digit clusters were merged or incorrectly split?
- Why might K-Means struggle to perfectly recover the 10 known digit classes, even with the correct K?
- What alternative clustering algorithm might better handle non-spherical or unevenly sized clusters, and why?

8. Conclusion (Template for Student Record)

In this experiment, K-Means clustering was implemented and evaluated on the Digits dataset across multiple values of K:

- Values of K tested: _____ , _____ , _____ , _____ , _____ , and others.
- Best-performing K based on silhouette score: _____ .
- Best-performing K based on the elbow method: _____ .

After preprocessing (feature scaling and, for visualization, PCA dimensionality reduction), K-Means was trained for each value of K and evaluated using inertia and silhouette score.

K (Clusters)	Inertia	Silhouette Score	Notes
2			
4			
6			
8			
10			

It was observed that _____ [student fills their observations, including which value of K best matched the true digit classes, how inertia and silhouette score trended with K, and what the cluster-center images and PCA scatter plot revealed about cluster quality]. The experiment achieved the course outcomes CO1, CO2, CO3, and CO4 by implementing, analysing, and comparing K-Means clustering results on real-world image data.

EXPERIMENT 18

Bootstrapping and Cross-Validation

Comparing Resampling Methods for Model Evaluation on the Iris Dataset

Course	Machine Learning (ML Lab)
Experiment	18 — Bootstrapping and Cross-Validation
Dataset	Iris Dataset (sklearn.datasets.load_iris)
Duration	3 Hours
Max. Marks	25 Marks

1. Aim

Implement bootstrapping and k-fold cross-validation as resampling methods to evaluate a classification model on the Iris dataset. The experiment covers loading and preprocessing the data, generating multiple bootstrap samples to estimate model performance, applying k-fold cross-validation, comparing performance metrics such as accuracy and F1-score obtained from each method, and discussing the advantages and disadvantages of each resampling approach.

2. Prerequisites

2.1 Theoretical Background

Before attending the lab, students must have a clear understanding of the following:

- Basics of supervised classification and simple train/test evaluation
- The concept of resampling methods and why a single train/test split can give an unreliable performance estimate
- Bootstrapping: sampling with replacement and the idea of out-of-bag (OOB) samples
- k-Fold cross-validation: partitioning data into k folds and training/testing across folds
- Stratified sampling and why it matters for classification datasets
- The bias-variance trade-off as it applies to model evaluation
- Evaluation metrics: accuracy, precision, recall, and F1-score
- Python programming: NumPy, Pandas, Matplotlib, Scikit-learn basics

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

Complete the following tasks before arriving at the lab:

- Explain the bootstrapping resampling method, including how samples are drawn and what out-of-bag samples are.
- Explain the k-fold cross-validation procedure, including how folds are formed and how the final metric is computed.
- State one advantage and one disadvantage of bootstrapping compared to a single train/test split.
- State one advantage and one disadvantage of k-fold cross-validation compared to a single train/test split.
- Explain why accuracy alone may be insufficient for evaluating a classifier, and how F1-score complements it.
- Explain how the choice of k in k-fold cross-validation affects the bias-variance trade-off of the performance estimate.
- Install required Python libraries: numpy, pandas, matplotlib, scikit-learn.

3. Theory

3.1 Why Resampling Methods?

A single train/test split produces only one estimate of model performance, which depends heavily on which data points happened to fall into the test set. Resampling methods reuse the available data multiple times — generating many training/testing combinations — to produce a distribution of performance estimates rather than a single number. This gives a more reliable picture of how well a model generalizes and how much its performance can vary.

3.2 Bootstrapping

Bootstrapping generates B bootstrap samples by drawing n instances with replacement from the original dataset of size n. Because sampling is with replacement, some instances appear multiple times in a bootstrap sample while others are left out entirely. The left-out instances are called out-of-bag (OOB) samples and are used to test the model trained on that bootstrap sample. This process is repeated B times, producing a distribution of B performance scores.

The probability that a specific instance is excluded from a single bootstrap sample of size n approaches the following limit as n grows large:

$$\mathbf{P(\text{not selected}) = (1 - 1/n)^n \approx e^{-1} \approx 0.368}$$

This means each bootstrap sample leaves out roughly 36.8% of the original data on average, which forms the out-of-bag test set for that iteration.

3.3 k-Fold Cross-Validation

k-Fold cross-validation splits the dataset into k equally sized folds. The model is trained k times: in each round, one fold is held out as the test set and the remaining k − 1 folds are used for training. The k resulting performance scores are then averaged to obtain the final cross-validation estimate, and their spread indicates how sensitive the model is to the particular training data used.

3.4 Stratified k-Fold Cross-Validation

For classification problems, stratified k-fold cross-validation ensures that each fold preserves approximately the same class proportions as the full dataset. This is good practice even for balanced datasets such as Iris (50 samples per class), since it prevents any fold from accidentally under-representing a class purely due to random splitting.

3.5 Evaluation Metrics

Metric	Formula	Interpretation
Accuracy	(Correct predictions) / Total	Overall fraction of correctly classified samples
Precision	TP / (TP + FP)	Of predicted positives, fraction actually correct
Recall	TP / (TP + FN)	Of actual positives, fraction correctly identified
F1-Score	2 · (Precision · Recall) / (Precision + Recall)	Harmonic mean of precision and recall

3.6 Bias-Variance Trade-off in Resampling

Resampling Choice	Effect on Estimate	Trade-off
Small k (e.g., k = 2 or 3)	Higher bias; each training fold is much smaller than the full dataset	Fast, but the performance estimate may be pessimistic
Moderate k (e.g., k = 5 or 10)	Good balance of bias and variance for most datasets	Widely used default; reasonable computation cost
Leave-One-Out CV (k =	Low bias, but high variance across	Very thorough, but computationally

n)	folds and high computational cost	expensive for larger datasets
Bootstrapping (B resamples)	Can have lower variance than a single split; OOB estimate is slightly pessimistic	Flexible sample size B, but samples overlap due to replacement

3.7 Comparing Bootstrapping and Cross-Validation

Aspect	Bootstrapping	k-Fold Cross-Validation
Sampling	With replacement; duplicates possible	Without replacement; each sample used once per round
Test set	Out-of-bag samples (~36.8% per iteration)	Held-out fold (1/k of the data per round)
Number of iterations	Flexible (commonly 100–1000)	Fixed at k (commonly 5 or 10)
Overlap between training sets	High — many bootstrap samples share instances	Lower — folds are disjoint
Typical use	Estimating variability of a statistic or metric	Model selection and generalization estimates

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / Google Colab / VS Code
- Libraries: numpy, pandas, matplotlib, scikit-learn
- Computer with minimum 4 GB RAM

5. Step-by-Step Procedure

Step 1: Import Libraries and Load Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.utils import resample
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score

# Load the Iris dataset (150 samples, 3 balanced classes, 4 features)
iris = load_iris()
X = iris.data
y = iris.target
print(X.shape, y.shape)
print(np.unique(y, return_counts=True))
```

Step 2: Exploratory Data Analysis

```
df = pd.DataFrame(X, columns=iris.feature_names)
df['species'] = pd.Categorical.from_codes(y, iris.target_names)
print(df.describe())
print(df['species'].value_counts())

pd.plotting.scatter_matrix(df.iloc[:, :4], c=y, cmap='viridis', figsize=(8, 8))
plt.suptitle('Iris Feature Pairwise Scatter Matrix')
plt.show()
```

Step 3: Data Preprocessing

```
# Feature scaling (helps gradient-based classifiers such as Logistic Regression)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Baseline single train/test split for reference
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y)
```

Step 4: Baseline Single Train/Test Split

```
baseline_model = LogisticRegression(max_iter=200, random_state=42)
baseline_model.fit(X_train, y_train)
y_pred_base = baseline_model.predict(X_test)

base_acc = accuracy_score(y_test, y_pred_base)
base_f1 = f1_score(y_test, y_pred_base, average='macro')
print(f'Single Split => Accuracy={base_acc:.4f}, F1(macro)={base_f1:.4f}')
```

Step 5: Implement Bootstrapping

```
n_iterations = 100
n_size = len(X_scaled)
boot_acc, boot_f1 = [], []

for i in range(n_iterations):
    # Sample n instances with replacement (bootstrap sample)
    indices = np.arange(n_size)
    train_idx = resample(indices, replace=True, n_samples=n_size, random_state=i)
    oob_idx = np.array([idx for idx in indices if idx not in set(train_idx)])
    if len(oob_idx) == 0:
        continue  # skip the rare iteration with no out-of-bag samples

    model = LogisticRegression(max_iter=200, random_state=42)
    model.fit(X_scaled[train_idx], y[train_idx])
    y_pred = model.predict(X_scaled[oob_idx])

    boot_acc.append(accuracy_score(y[oob_idx], y_pred))
    boot_f1.append(f1_score(y[oob_idx], y_pred, average='macro'))

print(f'Bootstrap (B={n_iterations}) => Accuracy={np.mean(boot_acc):.4f} ± {np.std(boot_acc):.4f}')
print(f'Bootstrap (B={n_iterations}) => F1(macro)={np.mean(boot_f1):.4f} ± {np.std(boot_f1):.4f}')
```

Step 6: Implement k-Fold Cross-Validation

```
model = LogisticRegression(max_iter=200, random_state=42)

cv_results = {}
for k in [5, 10]:
    skf = StratifiedKFold(n_splits=k, shuffle=True, random_state=42)
    acc_scores = cross_val_score(model, X_scaled, y, cv=skf, scoring='accuracy')
    f1_scores = cross_val_score(model, X_scaled, y, cv=skf, scoring='f1_macro')
    cv_results[k] = (acc_scores, f1_scores)
    print(f'{k}-Fold CV => Accuracy={acc_scores.mean():.4f} ± {acc_scores.std():.4f}, '
          f'F1(macro)={f1_scores.mean():.4f} ± {f1_scores.std():.4f}')
```

Step 7: Compare Resampling Methods

```
results_df = pd.DataFrame({
    'Method': ['Single Split', 'Bootstrap (B=100)', '5-Fold CV', '10-Fold CV'],
    'Mean Accuracy': [base_acc, np.mean(boot_acc), cv_results[5][0].mean(), cv_results[10][0].mean()],
    'Mean F1 (macro)': [base_f1, np.mean(boot_f1), cv_results[5][1].mean(), cv_results[10][1].mean()],
    'Std Dev (Accuracy)': [0.0, np.std(boot_acc), cv_results[5][0].std(), cv_results[10][0].std()],
})
print(results_df)
```

```
plt.figure(figsize=(8, 5))
plt.bar(results_df['Method'], results_df['Mean Accuracy'], color='steelblue')
plt.ylabel('Mean Accuracy')
plt.title('Comparison of Resampling Methods on Iris')
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```

Step 8: Visualize the Variability of Estimates

```
plt.figure(figsize=(8, 5))
plt.hist(boot_acc, bins=15, color='steelblue', edgecolor='white', alpha=0.8, label='Bootstrap
accuracies')
plt.axvline(cv_results[5][0].mean(), color='crimson', linestyle='--', label='5-Fold CV mean')
plt.axvline(cv_results[10][0].mean(), color='darkorange', linestyle='--', label='10-Fold CV mean')
plt.xlabel('Accuracy')
plt.ylabel('Frequency')
plt.title('Distribution of Bootstrap Accuracy vs Cross-Validation Means')
plt.legend()
plt.show()
```

6. Expected Results & Observations

Method	Expected Mean Accuracy	Expected Std Dev	Notes
Single Train/Test Split	0.93 – 1.00	Not applicable (one estimate)	A single lucky or unlucky split can over- or under-estimate true performance.
Bootstrap (B = 100)	0.93 – 0.97	0.02 – 0.05	Provides a full distribution of scores; OOB estimate is often slightly more pessimistic than CV.
5-Fold Cross-Validation	0.95 – 0.98	0.02 – 0.04	Good balance of bias and variance; computationally cheap for a small dataset like Iris.
10-Fold Cross-Validation	0.95 – 0.98	0.03 – 0.06	Lower bias than 5-fold, but each fold has fewer test samples, so fold-to-fold variance can be higher.

Note: Because Iris is a small, well-separated, and perfectly balanced dataset, most resampling methods will report high and fairly similar mean accuracy. The main practical difference between the methods shows up in the spread of the estimates: a single split gives no sense of variability at all, bootstrapping reveals variability through the distribution of out-of-bag scores, and cross-validation reveals variability across folds. Differences become far more pronounced on smaller, noisier, or more imbalanced datasets.

7. Discussion Questions

- Which resampling method gave the lowest variance in the estimated accuracy, and why do you think that happened?
- How did the bootstrap out-of-bag estimate compare to the k-fold cross-validation estimate in terms of mean accuracy?
- What are the computational trade-offs of running 100 bootstrap iterations compared to 5-fold or 10-fold cross-validation?
- Why can bootstrap samples contain duplicate instances, and how might this affect the trained model?

- In what scenario would you prefer leave-one-out cross-validation over k-fold cross-validation, despite its higher computational cost?
- Why is stratification important when performing k-fold cross-validation on a classification dataset, even one as balanced as Iris?
- If deploying a model in production, would you rely on a single train/test metric or a distribution of scores from resampling? Why?

8. Conclusion (Template for Student Record)

In this experiment, bootstrapping and k-fold cross-validation were implemented and compared as resampling methods on the Iris dataset:

- Resampling methods tested: single train/test split, bootstrap (B = 100), 5-fold CV, and 10-fold CV.
- Method with the lowest variance in accuracy: _____ .
- Method with the highest mean accuracy: _____ .

After preprocessing (feature scaling and a stratified baseline split), a Logistic Regression classifier was evaluated using each resampling method, reporting accuracy and F1-score (macro-averaged).

Method	Mean Accuracy	Mean F1 (macro)	Std Dev
Single Train/Test Split			
Bootstrap (B = 100)			
5-Fold Cross-Validation			
10-Fold Cross-Validation			

It was observed that _____ [student fills their observations, including which resampling method gave the most reliable estimate, how the mean and spread of accuracy compared across methods, and the practical trade-offs between bootstrapping and cross-validation in terms of computation and reliability]. The experiment achieved the course outcomes CO1, CO2, CO3, and CO4 by implementing, analysing, and comparing resampling-based model evaluation methods on real-world data.

EXPERIMENT 19

Bagging and Boosting Ensemble Methods

Implementing and Comparing Bagging and Boosting on the Titanic Dataset

Course	Machine Learning (ML Lab)
Experiment	19 — Bagging and Boosting Ensemble Methods
Dataset	Titanic Dataset (seaborn.load_dataset('titanic') / Kaggle Titanic CSV)
Duration	3 Hours
Max. Marks	25 Marks

1. Aim

Implement and compare Bagging and Boosting ensemble methods for classification on the Titanic dataset. The experiment covers loading and preprocessing passenger data, training a Bagging ensemble (using a Decision Tree as the base classifier) and a Boosting ensemble (using AdaBoost), evaluating both models using accuracy, precision, recall, and F1-score, and analysing how each ensemble strategy improves on a single base learner as well as their respective strengths and weaknesses.

2. Prerequisites

2.1 Theoretical Background

Before attending the lab, students must have a clear understanding of the following:

- Basics of supervised classification and Decision Trees
- The bias–variance tradeoff and how model complexity affects generalisation
- The concept of ensemble learning — combining multiple base learners into one model
- Bootstrap sampling (sampling with replacement) as used in Bagging
- Sequential re-weighting of misclassified samples as used in Boosting / AdaBoost
- Evaluation metrics: accuracy, precision, recall, F1-score, and the confusion matrix
- Handling missing values and encoding categorical features
- Python programming: NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn basics

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

Complete the following tasks before arriving at the lab:

- Explain what an ensemble method is and why combining multiple models often outperforms a single model.
- Explain the key difference between Bagging and Boosting in terms of how base learners are trained (parallel vs sequential).
- Explain how Bagging reduces the variance of a high-variance base learner such as a fully-grown Decision Tree.
- Explain how AdaBoost updates sample weights after each round and how the final prediction is combined.
- Explain precision, recall, and F1-score, and why accuracy alone can be misleading on the Titanic dataset.
- Describe one limitation of Bagging and one limitation of Boosting.
- Install required Python libraries: numpy, pandas, matplotlib, seaborn, scikit-learn.

3. Theory

3.1 Overview of Ensemble Learning

Ensemble learning combines the predictions of multiple base models to build a composite model that generalises better than any single model. The two most widely used ensemble families are Bagging, which trains base learners independently and in parallel to primarily reduce variance, and Boosting, which trains base learners sequentially to primarily reduce bias by focusing each new learner on the mistakes of the previous ones.

3.2 Bagging (Bootstrap Aggregating)

Bagging trains many copies of a base learner — typically a high-variance model such as an unpruned Decision Tree — on different bootstrap samples (random samples drawn with replacement) of the training data. Predictions from all base learners are then combined by majority vote for classification (or averaging for regression). Because each tree sees a slightly different subset of the data, individual errors tend to cancel out when the predictions are aggregated, which reduces the overall variance of the ensemble. Random Forest extends Bagging by additionally randomising the subset of features considered at each split.

3.3 Boosting

Boosting builds base learners sequentially, where each new learner is trained to correct the errors made by the ensemble so far. AdaBoost (Adaptive Boosting) is the classic algorithm: it starts with equal weights on all training samples, trains a weak learner (commonly a shallow 'decision stump'), increases the weights of the samples that were misclassified so the next learner focuses more on them, and repeats this process for a fixed number of rounds. The final prediction is a weighted majority vote in which more accurate learners are given a larger say in the outcome.

3.4 Bagging vs Boosting

Aspect	Bagging	Boosting
Training strategy	Parallel — base learners are independent	Sequential — each learner depends on the previous one
Primary goal	Reduce variance	Reduce bias (and often variance too)
Ideal base learner	High-variance, low-bias (deep/unpruned tree)	High-bias, low-variance (shallow tree / stump)
Combination rule	Simple / equal-weight majority vote	Weighted vote based on learner accuracy
Sensitivity to noise	Relatively robust to noisy data and outliers	More sensitive — misclassified noisy points get up-weighted
Risk of overfitting	Lower; more estimators rarely hurts	Higher with too many rounds on noisy data
Typical algorithms	Bagging, Random Forest	AdaBoost, Gradient Boosting, XGBoost

3.5 Evaluation Metrics

Given True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN):

$$\textbf{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

$$\textbf{Precision} = TP / (TP + FP) \qquad \textbf{Recall} = TP / (TP + FN)$$

$$\textbf{F1-Score} = 2 \times (Precision \times Recall) / (Precision + Recall)$$

On the Titanic dataset, the two classes (survived vs. did not survive) are imbalanced, so accuracy alone can hide poor performance on the minority class; precision, recall, and F1-score give a more complete picture of predictive quality.

3.6 Strengths and Weaknesses

Bagging:

- Strength: reduces variance and stabilises predictions; base learners can be trained in parallel; robust to noisy data and outliers.
- Weakness: does not reduce bias, so a systematically weak base learner will not improve much; requires training and storing many models; less interpretable than a single tree.

Boosting:

- Strength: reduces both bias and variance; often achieves higher accuracy than bagging with comparatively few, simple base learners.
- Weakness: sequential training cannot be easily parallelised; sensitive to noisy data and outliers since they receive increasing weight; more hyperparameters (learning rate, number of estimators) to tune; can overfit if boosted for too many rounds.

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / Google Colab / VS Code
- Libraries: numpy, pandas, matplotlib, seaborn, scikit-learn
- Computer with minimum 4 GB RAM

5. Step-by-Step Procedure

Step 1: Import Libraries and Load Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, AdaBoostClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report)

# Load the Titanic dataset (seaborn's built-in copy)
titanic = sns.load_dataset('titanic')
print(titanic.shape)
print(titanic.head())
```

Step 2: Exploratory Data Analysis

```
print(titanic.isnull().sum())

sns.countplot(x='survived', data=titanic)
plt.title('Survival Count')
plt.show()

sns.countplot(x='survived', hue='sex', data=titanic)
plt.title('Survival Count by Sex')
plt.show()

sns.countplot(x='survived', hue='pclass', data=titanic)
plt.title('Survival Count by Passenger Class')
plt.show()
```

Step 3: Data Preprocessing

```
df = titanic[['survived','pclass','sex','age','sibsp','parch','fare','embarked']].copy()
```

```
# Handle missing values
df['age'] = df['age'].fillna(df['age'].median())
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])

# Encode categorical variables
df['sex'] = LabelEncoder().fit_transform(df['sex'])
df['embarked'] = LabelEncoder().fit_transform(df['embarked'])

X = df.drop('survived', axis=1)
y = df['survived']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
print(X_train.shape, X_test.shape)
```

Step 4: Implement Bagging (Base Classifier: Decision Tree)

```
base_tree = DecisionTreeClassifier(random_state=42)

bagging_model = BaggingClassifier(
    estimator=base_tree, n_estimators=100, random_state=42)
bagging_model.fit(X_train, y_train)

y_pred_bagging = bagging_model.predict(X_test)

acc_bag  = accuracy_score(y_test, y_pred_bagging)
prec_bag = precision_score(y_test, y_pred_bagging)
rec_bag  = recall_score(y_test, y_pred_bagging)
f1_bag   = f1_score(y_test, y_pred_bagging)

print(f'Bagging => Accuracy={acc_bag:.4f}, Precision={prec_bag:.4f}, Recall={rec_bag:.4f},
F1={f1_bag:.4f}')
```

Step 5: Implement Boosting (AdaBoost)

```
stump = DecisionTreeClassifier(max_depth=1, random_state=42)

boosting_model = AdaBoostClassifier(
    estimator=stump, n_estimators=100, learning_rate=1.0, random_state=42)
boosting_model.fit(X_train, y_train)

y_pred_boosting = boosting_model.predict(X_test)

acc_boost  = accuracy_score(y_test, y_pred_boosting)
prec_boost = precision_score(y_test, y_pred_boosting)
rec_boost  = recall_score(y_test, y_pred_boosting)
f1_boost   = f1_score(y_test, y_pred_boosting)

print(f'Boosting => Accuracy={acc_boost:.4f}, Precision={prec_boost:.4f}, Recall={rec_boost:.4f},
F1={f1_boost:.4f}')
```

Step 6: Compare Performance Metrics

```
results = pd.DataFrame({
    'Model': ['Bagging', 'Boosting'],
    'Accuracy':  [acc_bag, acc_boost],
    'Precision': [prec_bag, prec_boost],
    'Recall':    [rec_bag, rec_boost],
    'F1-Score':  [f1_bag, f1_boost]
})
print(results)

results.set_index('Model').plot(kind='bar', figsize=(8, 5))
plt.title('Bagging vs Boosting Performance Comparison')
plt.ylabel('Score')
plt.ylim(0, 1)
```

```
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

Step 7: Visualize Confusion Matrices

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.heatmap(confusion_matrix(y_test, y_pred_bagging), annot=True, fmt='d',
            cmap='Blues', ax=axes[0])
axes[0].set_title('Bagging Confusion Matrix')
axes[0].set_xlabel('Predicted'); axes[0].set_ylabel('Actual')

sns.heatmap(confusion_matrix(y_test, y_pred_boosting), annot=True, fmt='d',
            cmap='Oranges', ax=axes[1])
axes[1].set_title('Boosting Confusion Matrix')
axes[1].set_xlabel('Predicted'); axes[1].set_ylabel('Actual')

plt.tight_layout()
plt.show()
```

Step 8: Compare Feature Importances

```
bag_importances = np.mean(
    [est.feature_importances_ for est in bagging_model.estimators_], axis=0)
boost_importances = boosting_model.feature_importances_

importance_df = pd.DataFrame({
    'Feature': X.columns,
    'Bagging': bag_importances,
    'Boosting': boost_importances
})
print(importance_df.sort_values('Boosting', ascending=False))
```

6. Expected Results & Observations

Model	Expected Accuracy	Expected Precision	Expected Recall	Expected F1-Score	Notes
Single Decision Tree	0.75 – 0.79	0.70 – 0.75	0.65 – 0.72	0.68 – 0.73	Baseline; prone to overfitting and high variance.
Bagging (100 trees)	0.78 – 0.82	0.75 – 0.80	0.70 – 0.76	0.72 – 0.78	Variance reduced vs. single tree; more stable predictions across runs.
Boosting — AdaBoost (100 stumps)	0.79 – 0.83	0.76 – 0.81	0.72 – 0.78	0.74 – 0.79	Often edges out bagging in accuracy/F1; more sensitive to outlier passengers.

Note: Exact values depend on the train/test split, random seed, and specific hyperparameters used. Boosting typically achieves a slightly higher accuracy and F1-score than Bagging on Titanic because sequential re-weighting lets it correct systematic mistakes (bias), whereas Bagging's improvement over a single tree comes mainly from variance reduction. Both ensembles should outperform a single, unconstrained Decision Tree, which tends to overfit the training data.

7. Discussion Questions

- Which ensemble method — Bagging or Boosting — achieved the higher accuracy and F1-score on the Titanic dataset in your run, and why might that be the case?
- How does bootstrap sampling in Bagging help reduce the variance of an unstable base learner such as a Decision Tree?
- How does AdaBoost's sample re-weighting mechanism help the ensemble focus on hard-to-classify passengers in later rounds?
- Why might Boosting be more sensitive to noisy or mislabeled data than Bagging?
- Between precision and recall, which metric matters more when predicting Titanic survival, and why?
- How did the feature importances differ between the Bagging and Boosting models? What does this suggest about how each ensemble weighs the input features?
- What would you expect to happen to Boosting's test performance if you kept increasing the number of estimators indefinitely? Would the same risk apply to Bagging?

8. Conclusion (Template for Student Record)

In this experiment, Bagging and Boosting ensemble methods were implemented and evaluated on the Titanic dataset:

- Base classifier used for Bagging: _____
- Number of estimators used: Bagging = _____, Boosting (AdaBoost) = _____
- Accuracy obtained — Bagging: _____ Boosting: _____
- Precision obtained — Bagging: _____ Boosting: _____
- Recall obtained — Bagging: _____ Boosting: _____
- F1-Score obtained — Bagging: _____ Boosting: _____

After preprocessing (handling missing values and encoding categorical features), both ensembles were trained on the same train/test split and compared using accuracy, precision, recall, and F1-score.

Model	Accuracy	Precision	Recall	F1-Score	Notes
Bagging					
Boosting (AdaBoost)					

It was observed that _____ [student fills their observations, including which method performed better on the Titanic dataset, how each ensemble improved on a single Decision Tree, and the practical strengths and weaknesses of Bagging versus Boosting they encountered]. The experiment achieved the course outcomes CO1, CO2, CO3, and CO4 by implementing, analysing, and comparing ensemble learning methods on real-world tabular data.

EXPERIMENT 20

Bias–Variance Tradeoff with Polynomial Regression

Investigating the Bias–Variance Tradeoff on the Boston Housing Dataset

Course	Machine Learning (ML Lab)
Experiment	20 — Bias–Variance Tradeoff (Polynomial Regression)
Dataset	Boston Housing Dataset (original StatLib source via fetch_openml('boston') / CSV)
Duration	3 Hours
Max. Marks	25 Marks

1. Aim

Investigate the bias–variance tradeoff using polynomial regression on the Boston Housing dataset. The experiment covers loading and preprocessing the housing data, fitting polynomial regression models of varying degree to a single predictive feature (or a small feature set), plotting training and validation error against polynomial degree, and analysing how model complexity drives the tradeoff between bias (underfitting) and variance (overfitting) and its effect on generalisation performance.

2. Prerequisites

2.1 Theoretical Background

Before attending the lab, students must have a clear understanding of the following:

- Basics of linear and polynomial regression
- The concepts of model bias and model variance
- Underfitting and overfitting, and how they relate to model complexity
- Train/validation/test splits and why validation error is used to judge generalisation
- Evaluation metrics for regression: Mean Squared Error (MSE) and R^2 score
- Feature scaling and polynomial feature expansion
- Python programming: NumPy, Pandas, Matplotlib, Scikit-learn basics

2.2 Pre-Lab Assignment (Assessed — Part of 7-Mark Preparation)

Complete the following tasks before arriving at the lab:

- Explain what bias and variance mean in the context of a machine learning model's predictions.
- Explain why a low-degree polynomial (e.g., degree 1) tends to underfit and a very high-degree polynomial tends to overfit.
- Explain the expected shape of the training-error curve and the validation-error curve as polynomial degree increases.
- Explain why training error alone cannot be used to select the best model complexity.
- Explain what Mean Squared Error (MSE) measures and why it is commonly used for regression evaluation.
- Describe one technique (other than choosing polynomial degree) that can help control overfitting, such as regularisation.
- Install required Python libraries: numpy, pandas, matplotlib, scikit-learn.

3. Theory

3.1 Overview of the Bias–Variance Tradeoff

The generalisation error of a supervised learning model can be decomposed into three components: bias, variance, and irreducible noise. Bias is the error introduced by approximating a real-world relationship with a simplified model; a model with high bias makes strong assumptions and tends to underfit, performing poorly on both training and validation data. Variance is the error introduced by the model's sensitivity to the particular training sample used; a model with high variance fits the training data very closely (including its noise) but generalises poorly, producing a large gap between training and validation error. Choosing an appropriate model complexity means balancing these two sources of error.

3.2 Polynomial Regression

Polynomial regression extends linear regression by adding powers of the input feature(s) — x, x^2, x^3, ..., x^d — as additional predictors, then fitting a linear model in this expanded feature space. The degree d controls the flexibility of the fitted curve: a degree-1 polynomial is a straight line (low complexity, high bias if the true relationship is non-linear), while a very high-degree polynomial can bend sharply to pass close to every training point (high complexity, high variance).

3.3 Underfitting and Overfitting

A polynomial of too low a degree underfits the data: it is too rigid to capture the true relationship between the features and the target, so both training error and validation error remain high. A polynomial of too high a degree overfits the data: it captures noise specific to the training set, so training error keeps decreasing (often toward zero) while validation error starts increasing again once the degree passes an optimal point. The degree that minimises validation error represents the best bias–variance tradeoff for the dataset.

3.4 Training Error vs Validation Error Curves

Plotting training error and validation error against increasing polynomial degree typically produces two characteristic curves: training error decreases monotonically (or nearly so) as degree increases, while validation error decreases initially, reaches a minimum, and then increases as the model starts to overfit. The point where the validation-error curve is lowest — and where the gap between the two curves starts widening beyond it — visually demonstrates the bias–variance tradeoff.

3.5 Bias vs Variance Summary

Aspect	High Bias (Underfitting)	High Variance (Overfitting)
Typical cause	Model too simple (e.g., low polynomial degree)	Model too complex (e.g., high polynomial degree)
Training error	High	Very low
Validation error	High, similar to training error	High, much larger than training error
Train–validation gap	Small	Large
Sensitivity to training data	Low — model barely changes with new samples	High — model changes substantially with new samples
Remedy	Increase model complexity / add features	Reduce complexity, add regularisation, or gather more data

3.6 Evaluation Metric: Mean Squared Error

For n samples with true values y and predictions ŷ, the Mean Squared Error is:

$$\mathbf{MSE = (1/n) \times \Sigma\ (y_i - \hat{y}_i)^2}$$

Lower MSE indicates predictions closer to the true values. Comparing training MSE and validation MSE across polynomial degrees is the standard way to visualise and diagnose the bias–variance tradeoff.

3.7 Impact on Model Performance

- Low-degree models (high bias): stable across different training sets, but systematically inaccurate — poor performance on both train and validation data.

- High-degree models (high variance): can fit training data almost perfectly, but predictions vary wildly with small changes in the training set and generalise poorly to new data.
- The goal of model selection is to find the sweet spot — the simplest model whose validation error is close to its minimum achievable value — rather than the model with the lowest training error.

4. Software & Hardware Requirements

- Python 3.8 or above
- Jupyter Notebook / Google Colab / VS Code
- Libraries: numpy, pandas, matplotlib, scikit-learn
- Computer with minimum 4 GB RAM

5. Step-by-Step Procedure

Step 1: Import Libraries and Load Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load the Boston Housing dataset from its original source
# (removed from sklearn.datasets in newer versions for ethical reasons;
# the original StatLib data is fetched directly instead)
data_url = 'http://lib.stat.cmu.edu/datasets/boston'
raw_df = pd.read_csv(data_url, sep='\\s+', skiprows=22, header=None)
data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
target = raw_df.values[1::2, 2]

columns = ['CRIM','ZN','INDUS','CHAS','NOX','RM','AGE','DIS','RAD',
           'TAX','PTRATIO','B','LSTAT']
boston = pd.DataFrame(data, columns=columns)
boston['MEDV'] = target
print(boston.shape)
print(boston.head())
```

Step 2: Exploratory Data Analysis

```
print(boston.describe())
print(boston.isnull().sum())

plt.figure(figsize=(6, 4))
plt.scatter(boston['LSTAT'], boston['MEDV'], alpha=0.5, color='steelblue')
plt.xlabel('LSTAT (% lower status of the population)')
plt.ylabel('MEDV (Median home value, $1000s)')
plt.title('MEDV vs LSTAT')
plt.show()
```

Step 3: Data Preprocessing

```
# Use LSTAT as the single predictor to clearly visualise polynomial fits;
# LSTAT has a strong non-linear relationship with MEDV
X = boston[['LSTAT']].values
y = boston['MEDV'].values

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.3, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
```

```
print(X_train.shape, X_val.shape)
```

Step 4: Implement Polynomial Regression for Varying Degrees

```
degrees = list(range(1, 16))   # degree 1 (linear) up to degree 15
train_errors = []
val_errors = []

for d in degrees:
    poly = PolynomialFeatures(degree=d)
    X_train_poly = poly.fit_transform(X_train_scaled)
    X_val_poly = poly.transform(X_val_scaled)

    model = LinearRegression()
    model.fit(X_train_poly, y_train)

    train_pred = model.predict(X_train_poly)
    val_pred = model.predict(X_val_poly)

    train_mse = mean_squared_error(y_train, train_pred)
    val_mse = mean_squared_error(y_val, val_pred)

    train_errors.append(train_mse)
    val_errors.append(val_mse)
    print(f'Degree {d:2d}  Train MSE={train_mse:8.3f}  Val MSE={val_mse:8.3f}')
```

Step 5: Plot Training vs Validation Error

```
plt.figure(figsize=(8, 5))
plt.plot(degrees, train_errors, marker='o', label='Training Error', color='steelblue')
plt.plot(degrees, val_errors, marker='o', label='Validation Error', color='crimson')
plt.xlabel('Polynomial Degree')
plt.ylabel('Mean Squared Error')
plt.title('Bias\u2013Variance Tradeoff: Training vs Validation Error')
plt.legend()
plt.grid(alpha=0.3)
plt.show()

best_degree = degrees[int(np.argmin(val_errors))]
print(f'Degree with lowest validation error: {best_degree}')
```

Step 6: Visualize Fitted Curves for Selected Degrees

```
sample_degrees = [1, 3, best_degree, 15]
x_plot = np.linspace(X_train_scaled.min(), X_train_scaled.max(), 200).reshape(-1, 1)

fig, axes = plt.subplots(2, 2, figsize=(11, 8))
for ax, d in zip(axes.flat, sample_degrees):
    poly = PolynomialFeatures(degree=d)
    X_train_poly = poly.fit_transform(X_train_scaled)
    model = LinearRegression().fit(X_train_poly, y_train)
    y_plot = model.predict(poly.transform(x_plot))

    ax.scatter(X_train_scaled, y_train, alpha=0.3, s=15, color='gray')
    ax.plot(x_plot, y_plot, color='crimson', linewidth=2)
    ax.set_title(f'Degree {d}')
    ax.set_xlabel('LSTAT (scaled)')
    ax.set_ylabel('MEDV')
plt.tight_layout()
plt.show()
```

Step 7: Evaluate the Selected Model with R^2 Score

```
poly_best = PolynomialFeatures(degree=best_degree)
X_train_best = poly_best.fit_transform(X_train_scaled)
X_val_best = poly_best.transform(X_val_scaled)
```

```
final_model = LinearRegression().fit(X_train_best, y_train)
train_r2 = r2_score(y_train, final_model.predict(X_train_best))
val_r2 = r2_score(y_val, final_model.predict(X_val_best))

print(f'Best Degree = {best_degree}')
print(f'Train R2 = {train_r2:.4f}, Validation R2 = {val_r2:.4f}')
```

6. Expected Results & Observations

Degree	Expected Train MSE	Expected Val. MSE	Notes
1 (linear)	High	High, ≈ train MSE	Underfits; straight line cannot capture the curved LSTAT–MEDV relationship.
2 – 3	Lower	Lower; near minimum	Good balance; captures the main curvature without chasing noise.
4 – 6	Lower still	Similar to or slightly above degree 2–3	Marginal gains on training data; validation error begins to flatten or creep up.
8 – 10	Very low	Noticeably higher than the minimum	Starts overfitting; curve wiggles to fit training noise.
12 – 15	Near zero	Sharply higher / unstable	Severe overfitting; curve oscillates wildly, especially near the edges of the data range.

Note: Exact MSE values depend on the train/validation split and random seed. The expected pattern is a training-error curve that decreases monotonically (or nearly so) with degree, and a validation-error curve shaped like a shallow 'U' — high at low degrees (bias-dominated error), reaching a minimum at a moderate degree, and rising again at high degrees (variance-dominated error). The fitted-curve plots should visually confirm this: the degree-1 line under-shoots the curvature in the data, the moderate-degree curve tracks the trend smoothly, and the degree-15 curve oscillates sharply, especially near the boundaries of the LSTAT range.

7. Discussion Questions

- At which polynomial degree did the validation error reach its minimum in your run, and how does the training error at that degree compare to the training error at degree 1 and at degree 15?
- Why does training error keep decreasing as polynomial degree increases, even when the model is clearly overfitting?
- What visual evidence in the fitted-curve plots (Step 6) indicates underfitting at low degrees and overfitting at high degrees?
- How would increasing the size of the training set be expected to affect the variance component of the error for high-degree polynomials?
- Why is validation error, rather than training error, the appropriate criterion for choosing the polynomial degree?
- Besides changing the polynomial degree, what other technique could reduce variance in a high-degree polynomial model without changing its degree?
- How would using multiple input features (instead of only LSTAT) change the way you would investigate the bias–variance tradeoff?

8. Conclusion (Template for Student Record)

In this experiment, polynomial regression models of varying degree were fitted on the Boston Housing

dataset to investigate the bias–variance tradeoff:

- Predictor feature(s) used: _____
- Range of polynomial degrees tested: _____ to _____
- Degree with the lowest validation MSE: _____
- Training MSE at the best degree: _____ Validation MSE at the best degree: _____
- Validation R^2 score at the best degree: _____

After preprocessing (train/validation split and feature scaling), polynomial regression was trained for each degree and evaluated using training and validation Mean Squared Error.

Degree	Train MSE	Validation MSE	Notes
1			
3			
5			
10			
15			

It was observed that _____ [student fills their observations, including which degree gave the best tradeoff between bias and variance, how the training and validation error curves diverged at high degrees, and what the fitted-curve plots revealed about underfitting versus overfitting]. The experiment achieved the course outcomes CO1, CO2, CO3, and CO4 by implementing, analysing, and visualising the bias–variance tradeoff on real-world housing data.